Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Jeu des Amazones – Projet C

Rapport final

Nicolas Caminade, Etudiant2, Etudiant3, Etudiant4, Etudiant5

9 avril 2026

1 Introduction

Le jeu des Amazones est un jeu de société combinatoire abstrait inventé par l'Argentin Walter Zamkauskas en 1988 et publié pour la première fois dans la revue *El Acertijo* en 1992 [1]. Il se joue traditionnellement sur un plateau de 10×10 cases et combine des éléments des échecs (le déplacement de la dame) et du jeu de Go (le contrôle du territoire).

1.1 Historique

Né en Argentine dans les années 1980, le Jeu des Amazones a d'abord circulé dans des cercles restreints d'amateurs de jeux abstraits avant d'être diffusé internationalement par la revue *El Acertijo* en 1992. Sa popularité auprès de la communauté informatique ne tardera pas à se manifester : dès la fin des années 1990, des chercheurs en intelligence artificielle s'emparent du jeu comme banc d'essai pour les algorithmes de recherche arborescente.

En 2005, l'équipe de Furtak démontre mathématiquement que résoudre ce jeu de manière générale est un problème PSPACE-complet [**?**], une classe de difficulté supérieure aux problèmes NP classiques, ce qui confirme définitivement que le jeu résiste à toute résolution exhaustive. Richard Lorentz établit ensuite, avec son programme *Invader*, que l'approche statistique du Monte Carlo Tree Search (MCTS) constitue à ce jour la stratégie la plus performante face à l'explosion combinatoire propre au jeu [**?**]. Depuis lors, des chercheurs intègrent des techniques de *Deep Learning* et de réseaux de neurones pour tenter de dépasser cet algorithme [**?**].

1.2 Règles

Le Jeu des Amazones se pratique sur un damier, traditionnellement de 10×10 cases. Chaque joueur dispose de quatre amazones, disposées symétriquement en début de partie selon une configuration initiale précise : les Blancs placent leurs pièces en (d1), (g1), (a4) et (j4), tandis que les Noirs occupent les cases (d10), (g10), (a7) et (j7). Cette disposition garantit un équilibre stratégique parfait entre les deux camps.

Objectif du jeu. La victoire revient au dernier joueur capable d'effectuer un coup légal. L'enjeu est donc de priver progressivement l'adversaire de tout espace de déplacement en combinant occupation du terrain et tirs de flèches. Contrairement aux jeux comme les échecs ou le Go, il n'existe pas de capture de pièce : le matériel reste constant tout au long de la partie.

Déroulement d'un tour. Chaque tour est impérativement composé de deux phases successives et indissociables. Dans un premier temps, le joueur choisit l'une de ses amazones et la déplace en ligne droite — verticalement, horizontalement ou en diagonale — vers une case libre. Ce mouvement est identique à celui de la dame aux échecs : l'amazone peut parcourir autant de cases qu'elle le souhaite dans la direction choisie, mais elle ne peut ni traverser ni s'arrêter sur une case occupée par une autre pièce (amie ou ennemie) ou par une flèche. Dans un second temps, depuis sa nouvelle position, cette même amazone doit obligatoirement tirer une flèche. La flèche obéit aux mêmes règles de trajectoire que l'amazone : elle se déplace en ligne droite dans l'une des huit directions possibles, sans pouvoir franchir d'obstacle. La case où la flèche atterrit est définitivement brûlée et inaccessible pour le reste de la partie.

Subtilités tactiques. Une règle souvent mal comprise par les débutants concerne le tir de la flèche : l'amazone peut tirer dans n'importe quelle direction autorisée depuis sa case d'arrivée, y compris en direction de la case qu'elle vient de quitter. Cette subtilité ouvre des possibilités tactiques riches, notamment la possibilité de s'auto-bloquer temporairement pour mieux circonscrire l'adversaire. De plus, la flèche ne peut traverser une case occupée par une amazone adverse, ce qui signifie que les pièces ennemies servent d'obstacles naturels aux tirs.

Fin de partie et variantes. Le jeu des Amazones ne connaît pas de match nul par répétition ou par épuisement de matériel : puisque chaque tour réduit irrémédiablement le nombre de cases disponibles, la partie se termine forcément par le blocage complet d'un joueur. En pratique, les parties se divisent souvent en deux phases distinctes : une ouverture stratégique où les amazones s'affrontent pour le contrôle des grandes zones du plateau, suivie d'une phase finale où chaque joueur maximise son territoire cloisonné. Bien que le format 10×10 soit le standard, des variantes sur plateau 6×6 ou 8×8 existent avec un nombre d'amazones adapté, permettant des parties plus rapides tout en conservant la richesse tactique du jeu.

FIGURE 1 – Évolution d'une partie : position initiale et premier coup [**?**][**?**]

1.3 Complexité

Le jeu des Amazones est d'une très grande complexité [**?**]. Le tableau ci-dessous emploie les abréviations Branch. pour « facteur de branchement » et Cmplx. pour « complexité ».

	Échecs	**Amazones**	**Go**
Facteur Branch. Initial	~ 20	~ 2176	~ 361
Facteur Branch. Moyen	~ 35	~ 374 joueur de départ ~ 299 deuxième joueur ~ 50 durant la fin de partie	~ 250
Nombre de tours moyen	~ 80	~ 84	~ 150
Cmplx. Arbre de jeu	$\sim 10^{123}$	$\sim 1.09 * 10^{212}$	$\sim 10^{360}$
Cmplx. Espaces d'états	de $\sim 3.7 * 10^{43}$ à $\sim 10^{52}$	$\sim 6.45 * 10^{40}$	$\sim 10^{170}$

TABLE 1 – Comparaison des complexités pour un plateau traditionnel (8×8 pour les échecs, 10×10 pour les Amazones, 19×19 pour le Go) [**?**][**?**]

Comme l'illustre ce tableau comparatif, la complexité des Amazones réside dans son facteur de branchement initial extrême, surpassant largement celui des échecs et comparable à celui du Go. Bien que la profondeur de jeu soit modérée, cette « largeur » excessive génère un arbre de jeu colossal (10^{212}), rendant toute exploration exhaustive impossible en raison de l'explosion combinatoire. Pour contourner cette explosion, les approches modernes s'appuient sur la sélectivité : solveurs de preuve comme *df-pn* [**?**], algorithmes de jeu utilisant du *Forward Pruning* [**?**], méthodes statistiques comme le MCTS via l'Upper Confidence bounds applied to Trees (UCT) [**?**], ou encore le Machine Learning.

2 Algorithmes et structures de données

2.1 Le cœur du jeu et l'Intelligence Artificielle

2.1.1 Représentation de l'état : les Bitboards

Pour garantir des performances optimales, nous avons modélisé le plateau via des *Bitboards* [**?**]. Le plateau pouvant atteindre une taille de 11×11 (121 cases), une simple variable de 64 bits était insuffisante. Nous avons donc conçu une structure `Bitboard` composée de deux entiers non signés (2×`uint64_t`), offrant 128 bits d'espace.

Cette structure permet de valider des coups ou de calculer des territoires en temps constant $\mathcal{O}(1)$ via des opérations bit-à-bit : ET, OU, XOR, décalages, *Pop Least-Significant-Bit*. La véritable puissance de cette structure s'exprime lors de la génération des trajectoires (*slides*). Dans nos heuristiques, nous distinguons deux types d'expansions : le ***King step***, qui simule une avancée d'une seule case dans les huit directions, et le ***Queen slide***, qui calcule la portée maximale d'une pièce sur ses lignes, colonnes et diagonales avant de rencontrer un obstacle.

Pour calculer ces *Queen slides* sans boucles itératives conditionnelles coûteuses, nous avons implémenté l'algorithme de **Kogge-Stone** [**?**]. Issu de l'architecture des additionneurs matériels, cet algorithme utilise le principe de propagation parallèle (*Parallel Prefix*) : il « pousse » simultanément tous les bits représentant nos pièces à travers un masque de cases vides, en utilisant des décalages bit-à-bit successifs et exponentiels (1, puis 2, puis 4, puis 8). L'intégralité des trajectoires possibles dans une direction donnée est ainsi calculée en seulement quatre instructions, couvrant largement la distance maximale de onze cases.

2.1.2 Analyse de Complexité des Opérations Bitboards

La majeur partie de nos opérations fondamentales sur les *Bitboards* s'exécutent en temps constant $\mathcal{O}(1)$. Il convient cependant de distinguer la complexité unitaire des instructions matérielles de leur application algorithmique :

— **Opérations basiques en $\mathcal{O}(1)$:** Les manipulations de masques (ET, OU, XOR) ainsi que les décalages de bits s'exécutent en un nombre fixe de cycles d'horloge. Le calcul du débordement entre nos deux entiers 64 bits (`low` et `high`) ajoute un surcoût mineur de quelques instructions processeur, mais la complexité temporelle globale reste strictement bornée et totalement indépendante de l'état du plateau.

— **Expansions spatiales en $\mathcal{O}(1)$:** C'est sur ce point que l'architecture Bitboard surpasse drastiquement les approches matricielles. La génération des trajectoires (*slides*), qu'il s'agisse d'un pas de Roi ou du calcul de la portée maximale d'une Reine, s'effectue en temps constant global. Notre implémentation de l'algorithme de Kogge-Stone déroule en effet une séquence stricte de 4 opérations de décalages exponentiels (1, 2, 4, 8) bornés par des masques. Le processeur calcule ainsi l'expansion de *toutes* les pièces de la même couleur de manière parfaitement simultanée, sans avoir recours à la moindre boucle conditionnelle.

— **Extraction et itération conditionnelle en $\mathcal{O}(k)$:** La seule véritable complexité variable de notre modèle intervient lors de la sérialisation des coups ou de l'évaluation des heuristiques. La fonction unitaire d'extraction `pop_lsb` s'exécute instantanément en $\mathcal{O}(1)$ grâce à un appel d'instruction matérielle spécifique. Toutefois, cette fonction est invoquée au sein d'une boucle. Cette boucle s'exécute donc exactement k fois, où k est le nombre strict de pièces actives évaluées. Nous garantissons ainsi une complexité algorithmique d'itération en $\mathcal{O}(k)$, un comportement infiniment plus performant qu'un parcours complet de la grille en $\mathcal{O}(N)$ inhérent aux tableaux bidimensionnels classiques.

2.1.3 L'approche Minimax et ses heuristiques

Le premier moteur d'IA repose sur l'algorithme Minimax, optimisé par l'élagage Alpha-Beta **[?]**. Pour pallier l'explosion combinatoire, nous avons enrichi cette base par deux mécanismes paramétrables. L'approfondissement itératif (*Iterative Deepening*) garantit le renvoi du meilleur coup trouvé à la profondeur maximale complétée si la limite de temps est atteinte. L'élagage anticipé (*Forward Pruning* **[?]**) restreint la largeur du faisceau de recherche en ne conservant que les coups offrant le plus de mobilité. Sans ces optimisations, le système bascule de manière transparente sur un Alpha-Beta exhaustif classique.

L'évaluation des nœuds feuilles constitue le cœur décisionnel de notre stratégie. Nous avons conçu et comparé trois fonctions d'évaluation offrant un compromis graduel entre rapidité et finesse mathématique.

La première approche est l'évaluation de mobilité **[?]**, axée sur la mobilité relative (`eval_me`), calcule rapidement la différence entre le nombre total de coups légaux disponibles pour les Blancs et les Noirs. En simulant une expansion de type *Queen slide* pour toutes les reines, l'algorithme dénombre les cases uniques atteignables. Soit M_p l'ensemble des cases atteignables par le joueur p, l'heuristique est définie par :

$$\mathcal{H}_{me} = |M_{blanc}| - |M_{noir}|$$

Cette légèreté calculatoire permet d'atteindre d'excellentes profondeurs d'exploration, mais l'évaluation demeure myope et peine à anticiper la fermeture structurelle des territoires.

La deuxième heuristique, dite de territoire relatif (`eval_rth`) **[?]**, évalue le contrôle spatial en déployant un parcours en largeur (BFS) simultané. Elle calcule la distance, en nombre de mouvements de Reine, séparant chaque case vide de la pièce la plus proche. Soit $\mathcal{E}$ l'ensemble des cases vides, et $d_W(x)$ (resp. $d_B(x)$) la distance minimale entre une case x et une reine blanche (resp. noire) :

$$\mathcal{H}_{rth} = \sum_{x \in \mathcal{E}} \Delta(x) \quad \text{avec} \quad \Delta(x) = \begin{cases} \min(d_B(x), C) - d_W(x) & \text{si } d_W(x) < d_B(x) \\ -(\min(d_W(x), C) - d_B(x)) & \text{si } d_B(x) < d_W(x) \\ 0 & \text{sinon} \end{cases}$$

Une distance infinie est plafonnée par une constante C (définie par $C =$ taille du plateau $+ 2$).

Notre modèle le plus abouti repose sur les territoires absolus combinés (`eval_qdkd`) **[?]**. Il simule l'appropriation du plateau par une expansion continue (*flood-fill*) traitée case par case. La force de cette heuristique réside dans sa résolution des conflits : si une case est atteinte par les deux joueurs à la même itération k, elle est déclarée contestée et agit comme un obstacle infranchissable. Soient $T_Q(p)$ et $T_K(p)$ les ensembles de cases conquises par le joueur p selon des déplacements respectifs de Reine et de Roi :

$$t_1 = |T_Q(Blanc)| - |T_Q(Noir)| \qquad t_2 = |T_K(Blanc)| - |T_K(Noir)|$$

Le calcul final applique une pondération mariant vision globale et contrôle local (choisi d'après les pistes de calcul de Lieberum **[?]**) :

$$\mathcal{H}_{qdkd} = 1.0 \times t_1 + 0.5 \times t_2$$

2.1.4 Monte Carlo Tree Search (MCTS) et apprentissage automatique

Le second moteur d'IA exploite le Monte Carlo Tree Search. Pour modéliser l'arborescence, nous avons opté pour une structure de données **LCRS** (*Left-Child Right-Sibling*) [**?**]. Ce choix découle directement du facteur de branchement initial supérieur à 2 000 coups possibles : utiliser des tableaux de pointeurs pour stocker les enfants de chaque nœud aurait provoqué une explosion de l'empreinte mémoire. Le choix de la structure LCRS (*Left-Child Right-Sibling*) répond à une contrainte majeure du jeu des Amazones : son facteur de branchement exceptionnellement élevé (entre 500 et 2 000 coups possibles par tour). Une approche classique, où chaque nœud pré-allouerait un tableau pointant vers tous ses enfants potentiels, saturerait la mémoire système de manière quasi instantanée.

À l'inverse, l'approche LCRS garantit une empreinte mémoire minimale et fixe. Chaque nœud ne contient que deux pointeurs : l'un vers son premier enfant, l'autre vers son frère immédiat. Cette optimisation spatiale implique un compromis temporel, car l'accès au k-ième enfant nécessite un parcours séquentiel en $O(k)$ cycles CPU.

Cependant, ce coût est largement justifié. Couplée à une instanciation dynamique des enfants — où un nœud n'est créé que lorsque le coup correspondant est réellement exploré — cette architecture empêche l'explosion mémoire tout en garantissant l'efficacité du moteur. Nous avons également enrichi ce MCTS d'un réseau de neurones. Dans ce mode, l'évaluation des nœuds non terminaux est combinée à l'inférence d'un modèle TensorFlow recevant en entrée un tenseur à quatre canaux généré par nos bitboards.

Politique de sélection UCT standard. Dans sa version classique, le MCTS guide la phase de sélection de l'arborescence à l'aide de la formule **UCT** (*Upper Confidence bounds applied to Trees*) [**?**]. Cette métrique applique les principes des bandits manchots à la recherche arborescente afin de trouver le compromis mathématique optimal entre l'exploitation des branches prometteuses et l'exploration des chemins peu visités. Pour un nœud enfant n, l'évaluation s'écrit :

$$\mathrm{UCT}(n) = Q(n) + c \cdot \sqrt{\frac{\ln(N(parent))}{N(n)}}$$

où $Q(n)$ représente la valeur moyenne empirique du nœud (le taux de victoire obtenu par les simulations aléatoires), $N(parent)$ et $N(n)$ désignent respectivement le nombre total de visites du nœud parent et de l'enfant évalué, et c est la constante d'exploration (théoriquement fixée à $\sqrt{2}$). Le premier terme favorise l'exploitation pure, tandis que le second garantit une exploration continue : la racine carrée croît de manière logarithmique avec les visites du parent, forçant l'algorithme à évaluer les enfants délaissés (où $N(n)$ est faible ou nul).

Variante PUCT pour l'apprentissage automatique. Lorsque le réseau de neurones est activé, la politique de sélection UCT classique est remplacée par la variante **PUCT** (*Predictor + Upper Confidence Bound*) [**?**]. Cette approche hybride combine l'exploration statistique d'UCT avec une évaluation par réseau de neurones pour guider plus efficacement la recherche.

La formule PUCT pour un nœud enfant n s'écrit :

$$\mathrm{PUCT}(n) = \frac{Q_{\mathrm{rollout}}(n) + Q_{\mathrm{NN}}(n)}{2} + c_{\mathrm{puct}} \cdot \frac{\sqrt{N(parent)}}{1 + N(n)}$$

où $Q_{\mathrm{rollout}}(n)$ est le taux de victoire empirique obtenu par les simulations Monte Carlo, $Q_{\mathrm{NN}}(n)$ est la probabilité de victoire prédite par le réseau de neurones, $N(parent)$ et $N(n)$ sont respectivement les nombres de visites du parent et de l'enfant, et c_{puct} est une constante d'exploration (par défaut 1.5).

Le réseau de neurones utilisé est un modèle de régression linéaire simple entraîné sur des parties auto-générées. Il reçoit en entrée un tenseur de dimension $11 \times 11 \times 4$ représentant :

— Canal 0 : positions des reines blanches (1 si présente, 0 sinon)
— Canal 1 : positions des reines noires
— Canal 2 : positions des flèches
— Canal 3 : masque de validité du plateau (1 pour les cases valides)

Cette architecture à quatre canaux permet au modèle de capturer simultanément la configuration matérielle et la topologie du plateau. Lors de la sélection, si l'évaluation neuronale d'un nœud n'a pas encore été calculée (`ai_eval` ≤ -2.0), le tenseur correspondant est généré à la volée et soumis au modèle TensorFlow. La valeur retournée est ensuite mise en cache dans le nœud pour éviter les inférences redondantes.

2.2 Réseau : structures de données et protocole

La mise en place du mode multijoueur a nécessité de résoudre deux contraintes fondamentales de la programmation réseau : la nature orientée flux du protocole TCP et la nécessité de maintenir un état cohérent pour chaque connexion asynchrone. Cette section présente les structures de données conçues pour répondre à ces exigences.

2.2.1 Le problème du *Framing* TCP

Contrairement à UDP qui fonctionne par datagrammes délimités, TCP est orienté flux : l'émetteur peut envoyer trois messages de 100 octets, mais le récepteur pourrait recevoir un bloc de 250 octets suivi d'un fragment de 50 octets. Ce phénomène, appelé *TCP Framing*, rend impossible la lecture directe de messages individuels sans mécanisme de délimitation.

Choix du délimiteur. Nous avons opté pour un délimiteur de fin de message (`'\n'`) plutôt qu'un préfixe de longueur. Ce choix s'inscrit dans la tradition des protocoles applicatifs textuels (SMTP, POP3, IRC) et présente deux avantages : il permet le débogage à l'œil nu via des outils standards (`netcat`, `tcpdump`), et il évite les problèmes de désynchronisation si un octet de longueur est corrompu — une erreur dont la récupération serait impossible sans fermer la connexion.

Structure de tampon. Pour accumuler les fragments TCP jusqu'à réception d'un message complet, chaque connexion dispose de son propre tampon linéaire de 4 096 octets. Ce dimensionnement (environ 2,7 fois le MTU Ethernet standard de 1 500 octets) garantit qu'au moins deux messages complets peuvent être mis en file simultanément, absorbant ainsi les pics de trafic sans saturation.

Algorithme d'extraction. L'extraction des messages s'effectue en trois phases distinctes. La première phase consiste en l'**accumulation** : les octets bruts reçus via `recv()` sont concaténés à la fin du tampon. La deuxième phase est la **recherche** : une recherche linéaire localise le premier caractère de fin de ligne dans le tampon. La troisième phase est la **consommation** : le message extrait est copié vers un tampon de sortie, puis les octets restants sont décalés vers le début du tampon via `memmove()`, préservant ainsi les fragments du message suivant. Cette dernière opération, bien que coûteuse en temps ($\mathcal{O}(n)$ où n est la taille des données restantes), reste négligeable face aux latences réseau et garantit une gestion mémoire sans fragmentation.

2.2.2 Protocole applicatif et sérialisation

Au-dessus de la couche de framing, nous avons défini un protocole applicatif textuel structuré autour d'une unité atomique : le `Packet`.

Choix du format texte. Le format ASCII a été préféré au binaire pour trois raisons. Premièrement, il élimine les problèmes de boutisme (*endianness*) lors de la communication entre architectures hétérogènes (x86, ARM). Deuxièmement, il facilite le débogage sans outils spécialisés : un simple `netcat` suffit à observer les échanges. Troisièmement, il permet une évolution du protocole par ajout de nouveaux types de messages sans rétrocompatibilité binaire complexe.

Format de sérialisation. Chaque paquet suit le format `"TYPE|PAYLOAD\n"`, où `TYPE` est un entier identifiant le type de message, le caractère `'|'` sert de séparateur unique entre le type et la charge utile, et le caractère `'\n'` permet à la couche de framing de détecter la fin du message. Cette conception garantit un parsing déterministe en $\mathcal{O}(m)$ où m est la longueur du message.

Le protocole définit 27 types de paquets répartis en trois catégories : les messages client vers serveur (préfixés `C2S_`), les messages serveur vers client (préfixés `S2C_`), et les messages bidirectionnels. Le tableau 23 en annexe présente l'ensemble des types avec leur rôle respectif.

2.2.3 Structures de contexte et gestion des connexions

Modélisation d'un client connecté. Chaque client est représenté par une structure à taille fixe regroupant l'ensemble des informations nécessaires à sa gestion : le descripteur de socket, son tampon TCP dédié, son pseudo, son statut courant dans la machine à états (voir section **??**), l'identifiant de sa partie en cours, l'horodatage de sa dernière activité, et les métadonnées d'invitation pendante. Cette encapsulation permet de traiter chaque client comme une entité autonome sans variables globales partagées.

Pool d'objets pré-alloués. Une décision architecturale majeure concerne l'allocation mémoire : les 64 slots clients et les 32 slots de parties sont alloués **une seule fois** au démarrage du serveur. Aucune allocation dynamique n'intervient lors des connexions ou déconnexions : un client qui se connecte occupe un slot précédemment marqué invalide, et sa déconnexion remet simplement ce marqueur. Ce patron de *pool d'objets* présente trois avantages : il élimine la fragmentation mémoire sur les serveurs à longue durée de vie, il garantit un temps de traitement constant par connexion ($\mathcal{O}(1)$ pour trouver un slot libre dans le pire cas via parcours linéaire borné), et il rend l'empreinte mémoire totale déterministe et prévisible dès le démarrage.

Correspondance d'index pour un accès en temps constant. Une optimisation critique réside dans l'alignement des tableaux de surveillance et de métadonnées : l'index i dans le tableau `fds[]` utilisé par `poll()` correspond **exactement** à l'index i dans le tableau `clients[]`. Cette correspondance bijective permet de passer en $\mathcal{O}(1)$ du descripteur de fichier signalé par le système d'exploitation aux informations complètes du joueur (pseudo, statut,

partie en cours), sans recourir à une recherche linéaire ou à une table de hachage. L'index 0 est réservé à la socket d'écoute du serveur, les index 1 à 64 correspondent aux clients connectés.

2.3 Contrôleurs : structures de données et contextes

L'architecture de l'application repose sur deux contrôleurs distincts, chacun encapsulant l'intégralité de son état dans une structure de contexte dédiée. Ce patron d'encapsulation élimine les variables globales et facilite le passage d'état entre les différents gestionnaires d'événements.

2.3.1 Le contexte CLI `CliContext`

Le contrôleur en ligne de commande centralise l'ensemble des informations nécessaires à la gestion d'une session de jeu. La structure `CliContext` regroupe cinq catégories de données :

```c
typedef struct {
  // etat du jeu
  GameState *current_game;
  game_config_t *config;

  // etat de navigation
  bool game_running;
  bool in_main_menu;
  bool in_network_match;
  bool is_away;

  // Role en partie reseau
  PlayerColor my_role;

  // Instances IA (une par couleur)
  AI_Interface ai_white;
  bool ai_white_initialized;
  AI_Interface ai_black;
  bool ai_black_initialized;

  // Gestionnaire reseau
  NetworkManager *net_manager;
  int last_invited_id;

  // Gestion du temps
  time_t turn_start_time;
  int timeout_limit;
  bool timeout_sent;
} CliContext;
```

Listing 1 – Structure du contexte CLI

Cette structure permet au contrôleur de : (1) bloquer certaines commandes selon le mode actif (par exemple, empêcher `undo` en partie réseau), (2) router les coups vers le réseau ou l'IA selon la configuration, et (3) gérer le timeout de tour en mode synchronisé.

2.3.2 Le contexte GUI `AppContext`

De manière symétrique, le contrôleur graphique encapsule son état dans une structure `AppContext` adaptée à GTK4. Cette structure contient les références aux widgets principaux, l'état de la partie, et les métadonnées de sélection pour le glisser-déposer :

```c
typedef struct {
  // etat du jeu
  GameState *gameState;
  game_config_t *gameConfig;

  // Widgets GTK principaux
  GtkWidget *mainWindow;
  GtkWidget *mainStack;
  GtkWidget **boardCells;  // Tableau 1D aplati (y * size + x)

  // etat de la phase de tour
  GuiPhase currentPhase;   // PHASE_MOVE_QUEEN ou PHASE_SHOOT_ARROW

  // Selection en cours (pour le glisser-deposer)
  Position selectedQueen;
  Position tempQueenDst;

  // Raccourcis clavier personnalisables
```

```
  KeyBindings keyBindings;

  // Reseau (non connecte en GUI dans cette version)
  NetworkManager *netManager;
} AppContext;
```

Listing 2 – Structure du contexte GUI (extrait)

Le choix d'un tableau unidimensionnel pour `boardCells` (indexé par `y * size + x`) simplifie les conversions de coordonnées et évite la fragmentation mémoire inhérente aux tableaux de pointeurs.

2.4 Moteur de jeu : structures de données

Afin de garantir un moteur de jeu rapide et évolutif tout en anticipant les besoins de calcul intensif de l'intelligence artificielle, une attention particulière a été portée à la conception des structures de données de bas niveau.

2.4.1 Modélisation matricielle de l'état du jeu

L'état global de la partie est encapsulé dans une structure unique `GameState`. La représentation structurelle de base du plateau est stockée sous forme d'un tableau dynamique à deux dimensions (matrice de type `CellContent** board`). Chaque cellule contient une énumération définissant son état (`CONTENT_EMPTY`, `CONTENT_WHITE_QUEEN`, `CONTENT_ARROW`, etc.). Cette modélisation matricielle offre une lisibilité optimale pour l'affichage et permet un accès spatial en temps constant $\mathcal{O}(1)$. Cependant, elle montre de fortes limites pour l'exploration de l'arbre par l'IA : le parcours itératif d'une matrice 2D deviendrait un goulot d'étranglement lors des millions de simulations nécessaires, d'où le recours aux bitboards décrits plus haut.

2.4.2 Architecture Bitboard

Le moteur maintient et consolide l'état du jeu à travers trois `Bitboards` distincts, extraits dynamiquement depuis le `GameState` : `bb_white` pour la position des reines blanches, `bb_black` pour les reines noires, et `bb_arrows` pour l'ensemble des flèches tirées. Cette représentation secondaire parallèle est indispensable pour l'IA et les fonctions d'évaluation, tandis que la représentation matricielle reste utilisée pour l'affichage et la validation unitaire des règles.

2.5 Moteur de jeu : algorithmes

2.5.1 Algorithme de validation des trajectoires (Raycasting)

La règle principale des Amazones stipule qu'aucune pièce ne peut franchir un obstacle. Pour valider mathématiquement un coup saisi par un joueur, nous avons développé l'algorithme `is_path_clear` fonctionnant sur le principe du lancer de rayon (*Raycasting*) géométrique.

L'algorithme calcule d'abord le vecteur de déplacement $\vec{v} = (\Delta x, \Delta y)$ entre le point de départ $A(x_1, y_1)$ et le point d'arrivée $B(x_2, y_2)$, puis vérifie que le mouvement respecte la contrainte de la dame aux échecs, c'est-à-dire que $\Delta x = 0$, ou $\Delta y = 0$, ou $|\Delta x| = |\Delta y|$. Il détermine ensuite un pas géométrique valant -1, 0 ou 1 pour chaque axe, via la formule $step_x = \text{sign}(\Delta x)$ et $step_y = \text{sign}(\Delta y)$. Une boucle itérative parcourt alors le segment point par point : si le `Bitboard` consolidé des obstacles possède un bit à `1` à l'indice courant, la trajectoire est bloquée et la fonction retourne immédiatement « Faux ».

2.5.2 Piles d'historique et algorithme d'annulation (Undo/Redo)

La mécanique d'annulation de coups repose sur des structures de piles (`MoveStack`) implémentant le principe LIFO. Pour accélérer la lecture par le processeur et éviter la fragmentation mémoire, nous avons opté pour des tableaux dynamiques pré-alloués formant un seul bloc continu.

Un coup est atomique : il modifie simultanément la position d'une reine et place une flèche. La structure `Move` stocke ces trois données (case source de la reine, case destination de la reine, case destination de la flèche). L'algorithme d'annulation exécute la mutation inverse : il vide la case de la flèche, replace la reine à sa position d'origine, décrémente le compteur de tours, bascule le joueur courant, et empile immédiatement le `Move` dans la pile de redo pour autoriser une potentielle action « Refaire ».

2.6 Interface graphique : structures de données

L'interface graphique s'appuie sur la boîte à outils GTK4 [2], qui suit un paradigme orienté objet.

2.6.1 Le contexte applicatif `GuiContext`

De façon symétrique à l'architecture de la couche terminal, toute l'information d'état de l'interface graphique est regroupée dans un objet global `GuiContext`. Ce patron d'encapsulation (*Context Object*) évite les variables globales et facilite le passage d'état entre les différents gestionnaires d'événements.

Cette structure centralise tous les sous-systèmes : la configuration du jeu, l'état courant de la partie (`GameState`), le conteneur principal de la fenêtre, mais également le moteur d'IA et le gestionnaire réseau (`NetworkManager`). Le regroupement de ces instances permet au contrôleur graphique d'orchestrer des parties locales, contre l'IA, ou en multijoueur réseau de façon transparente.

Le plateau de jeu visuel est représenté par un tableau unidimensionnel de conteneurs de taille $n \times n$. Ce choix d'aplatissement (indexé par `y * size + x`) simplifie les calculs de conversion de coordonnées matricielles et évite la fragmentation en mémoire.

2.6.2 L'automate des phases de jeu

Un tour aux Amazones est indivisible : déplacer une reine puis tirer une flèche constitue une seule action logique. Pour modéliser cette contrainte dans l'interface graphique, nous avons défini une machine à états finis régie par deux statuts : `PHASE_MOVE_QUEEN` et `PHASE_SHOOT_ARROW`. Ce mécanisme garantit que l'interface est un automate déterministe : un événement de clic ou de glisser-déposer n'est traité que si l'état courant correspond à l'action attendue par les règles du jeu.

2.6.3 Les raccourcis clavier `KeyBindings`

Une structure `KeyBindings` stocke sous forme de chaînes de caractères (au format d'accélérateurs logiciels, ex : `"<Control>n"`) les raccourcis clavier de l'application, modifiables en direct. À chaque modification, le contrôleur de raccourcis de la fenêtre est dynamiquement détruit puis recréé, garantissant une cohérence stricte entre la vue des paramètres et le déclenchement des actions.

3 Architecture du projet

3.1 Découpage en modules

3.1.1 Diagramme de dépendances

L'application suit une architecture en couches respectant strictement le principe de séparation des responsabilités. La figure 2 illustre les dépendances entre modules.

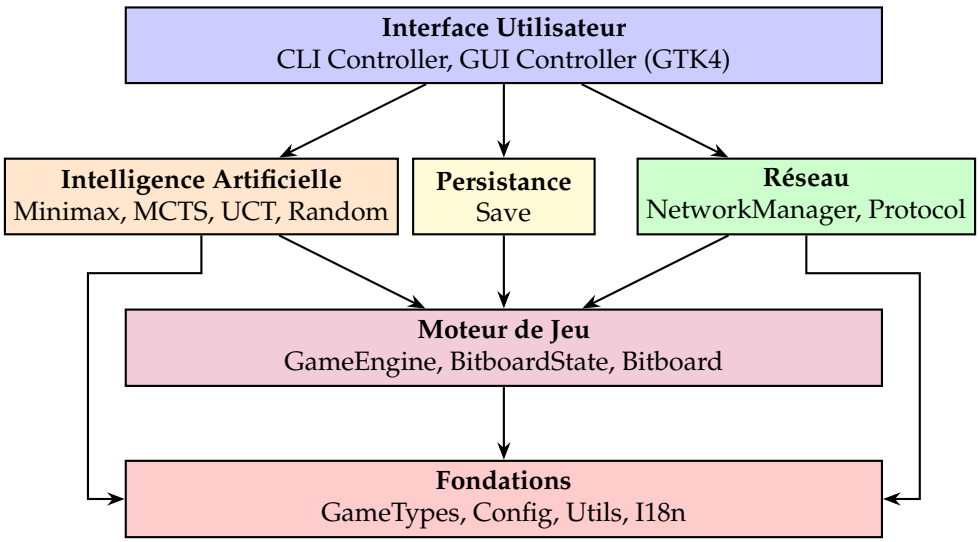


FIGURE 2 – Architecture en couches du projet. Les flèches représentent les dépendances : chaque module dépend uniquement des couches inférieures, garantissant testabilité et maintenabilité.

Le moteur de jeu constitue le **noyau pur** de l'application : totalement indépendant de l'affichage et du réseau, il expose une API pour la manipulation de l'état du jeu. Les modules d'IA interrogent ce moteur via l'interface `BitboardState` pour des calculs haute performance, tandis que les contrôleurs manipulent le `GameState`. Le réseau et l'IA communiquent avec l'interface via un système de **callbacks**, évitant ainsi toute dépendance cyclique.

3.2 Architecture du module IA

3.2.1 Le patron Factory et la table virtuelle

Le système d'IA repose sur deux abstractions complémentaires : une **usine de création** (`ai_factory.c`) et une **interface polymorphe** (`AI_Interface`). Cette architecture permet d'instancier dynamiquement différents types d'IA à partir de chaînes de caractères, sans couplage statique entre le contrôleur et les implémentations concrètes.

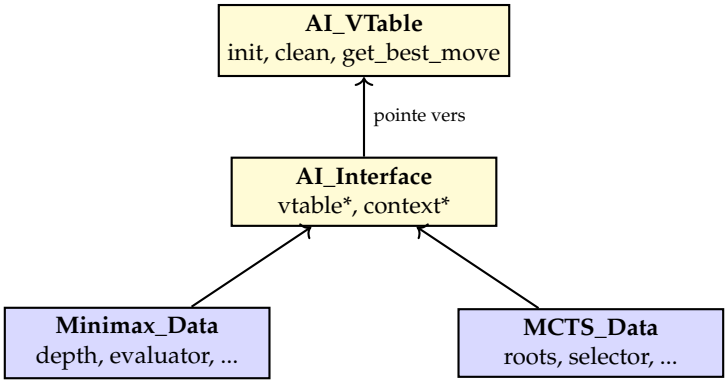


FIGURE 3 – Diagramme UML simplifié du système d'IA. Chaque IA concrète implémente les trois méthodes de la VTable et stocke son état privé dans `context`.

L'usine maintient un registre statique des IA disponibles :

```
static const AI_RegistryEntry AI_REGISTRY[] = {
    { "random", create_random_ai },
    { "minimax", create_minimax_ai },
    { "iterative", create_minimax_ai }, // alias
    { "mcts", create_mcts_ai }
};
```

Listing 3 – Extrait du registre de l'usine

Lors de l'appel à `create_ai("mcts", params)`, l'usine :

1. Recherche "mcts" dans le registre
2. Invoque `create_mcts_ai(&ai)` pour initialiser la VTable
3. Appelle `ai.vtable->init(&ai, &params)` pour construire le contexte
4. Retourne l'interface complète

Cette indirection permet d'ajouter de nouvelles IA sans modifier le code appelant. Par exemple, ajouter une IA basée sur un solveur de preuve nécessite uniquement :

— Créer `src/ai/solver/solver.c` avec les fonctions `solver_init`, `solver_clean`, `solver_get_best_move`
— Ajouter `{ "solver", create_solver_ai }` au registre

3.2.2 Injection de dépendances par ConfigMap

Les paramètres de configuration sont transmis sous forme de dictionnaire clé-valeur dynamique :

```
Param params[] = {
    { "depth", "5" },
    { "evaluator", "qd&kd" },
    { "forward_pruning", "1" }
};
ConfigMap cfg = { 3, params };
AI_Interface ai = create_ai("minimax", cfg);
```

Listing 4 – Exemple d'utilisation du ConfigMap

La macro générique `check_param` utilise `_Generic` (C11) pour dispatcher automatiquement vers la fonction de parsing appropriée selon le type de la cible :

```
int depth = 3;
bool pruning = false;
const char* eval = "me";

check_param(params[0], "depth", &depth);          // -> int
check_param(params[1], "evaluator", &eval);      // -> const char**
check_param(params[2], "forward_pruning", &pruning); // -> bool
```

Cette approche élimine le besoin de structures de configuration rigides et permet l'ajout de nouveaux paramètres sans casser la compatibilité binaire.

3.2.3 Découplage évaluateurs et sélecteurs

Les heuristiques d'évaluation (Minimax) et les stratégies de sélection (MCTS) sont également découplées par des interfaces :

```c
typedef float (*Evaluator)(const BitboardState* state);

Evaluator get_evaluator(EvaluatorType type) {
    switch (type) {
        case EVAL_ME: return eval_me;
        case EVAL_RTH: return eval_rth;
        case EVAL_QD_KD: return eval_qdkd;
        default: return eval_me;
    }
}
```

Listing 5 – Interface des évaluateurs

Les sélecteurs MCTS suivent un patron similaire avec une VTable dédiée (`ISelector_VTable`) permettant de basculer entre UCT et PUCT sans recompiler le cœur de MCTS.

3.3 Le moteur de jeu

Le moteur de jeu a été architecturé comme le modèle pur de l'application. Il est totalement indépendant du système d'entrée/sortie : son code interne ne contient aucune instruction d'affichage sur la sortie standard, n'écrit pas dans des fichiers, et ne tente aucune lecture asynchrone sur le clavier. Il communique avec l'extérieur exclusivement via une API publique définie dans son en-tête.

Grâce à cette architecture de composants isolés, le même moteur de jeu peut arbitrer une partie locale dans le terminal, valider des coups asynchrones sur le serveur réseau, ou simuler des parties virtuelles pour l'IA, sans nécessiter la moindre modification de sa structure interne.

3.4 Architecture du module réseau

L'architecture du module réseau repose sur trois piliers conceptuels : une façade centralisée masquant la complexité des sockets, un modèle événementiel mono-threadé basé sur le multiplexage, et un système de rappels découplant le cœur réseau des interfaces utilisateur. La figure 4 illustre l'organisation générale de ces composants.

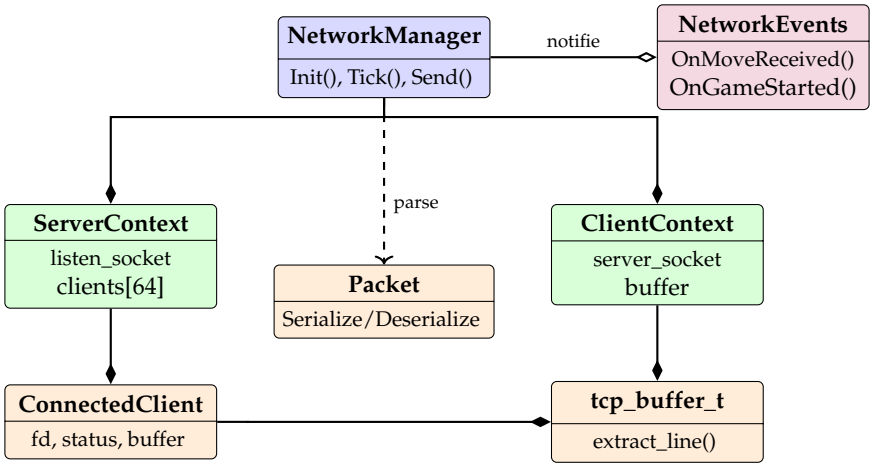


FIGURE 4 – Diagramme de composition du `NetworkManager`. La façade possède au plus un contexte serveur **ou** client (exclusion mutuelle). Chaque client embarque son propre tampon TCP. Légende : ♦ composition, ◊ agrégation, trait pointillé = dépendance.

3.4.1 La façade `NetworkManager`

Principe architectural. Le module réseau expose une unique structure publique — `NetworkManager` — qui encapsule l'intégralité de la logique de communication. Les couches supérieures (CLI, GUI) n'ont jamais besoin de manipuler directement les descripteurs de fichiers, les structures d'adresses réseau, ni les tampons de framing. Ce masquage strict de l'information garantit que toute évolution interne (passage à TLS, changement de protocole) n'impacte pas le code client.

L'initialisation de la façade illustre cette encapsulation :

```c
void NetworkManager_Init(NetworkManager *manager, NetworkEvents callbacks) {
  manager->current_mode = NET_MODE_NONE;
  manager->server_ctx = NULL;
  manager->client_ctx = NULL;
  manager->callbacks = callbacks;  // Injection des rappels
}
```

Listing 6 – Initialisation du NetworkManager

Dualité serveur/client. Le `NetworkManager` peut fonctionner en trois modes mutuellement exclusifs : inactif, serveur ou client. En mode serveur, il alloue un `ServerContext` contenant la socket d'écoute, le tableau des 64 slots clients avec leurs tampons TCP respectifs, le tableau des 32 parties actives, et le contexte de découverte UDP. En mode client, il alloue un `ClientContext` minimaliste contenant uniquement la socket de connexion au serveur et son tampon de réception. Cette ségrégation mémoire évite toute confusion entre les rôles et permet une libération ciblée des ressources.

3.4.2 Le multiplexage asynchrone avec `poll()`

Choix du modèle événementiel. Pour gérer simultanément jusqu'à 64 connexions sans threads, nous avons opté pour le multiplexage d'entrées/sorties via l'appel système `poll()`. Trois alternatives existaient : `select()`, limité à 1 024 descripteurs et utilisant des masques de bits peu pratiques ; `epoll()`, plus performant mais spécifique à Linux ; et `poll()`, offrant une interface portable POSIX avec des performances adaptées à notre charge. Le modèle monothreadé élimine par construction les conditions de course et la nécessité de verrous, simplifiant considérablement le raisonnement sur la correction du code.

Boucle non bloquante. La fonction `NetworkManager_Tick()`, appelée à chaque itération de la boucle principale, invoque `poll()` avec un timeout nul. Ce paramétrage rend l'appel instantanément retourné : si des données sont disponibles, elles sont traitées ; sinon, le contrôle revient immédiatement à l'appelant. Cette conception permet à l'interface de continuer à traiter d'autres événements (saisie clavier, rafraîchissement graphique) entre deux cycles réseau.

```c
static void handle_server_tick(NetworkManager *manager, ServerContext *ctx) {
  int poll_count = poll(ctx->fds, MAX_CLIENTS + 1, POLL_TIMEOUT_IMMEDIATE);

  if (poll_count > 0) {
    if (ctx->fds[LISTEN_SOCKET_INDEX].revents & POLLIN)
      handle_server_new_connection(ctx);

    for (int i = FIRST_CLIENT_INDEX; i <= MAX_CLIENTS; i++) {
      if (ctx->fds[i].fd != INVALID_SOCKET &&
          (ctx->fds[i].revents & (POLLIN | POLLHUP | POLLERR)))
        handle_server_client_data(manager, ctx, i);
    }
  }
}
```

Listing 7 – Tick serveur avec poll() non bloquant

Double multiplexage dans la boucle CLI. La boucle principale du CLI illustre une application avancée de ce modèle : elle doit surveiller simultanément deux sources d'événements indépendantes — le réseau et le clavier. Plutôt que de créer deux threads ou de bloquer sur l'une des sources, nous utilisons deux appels `poll()` distincts à chaque itération : le premier interroge les sockets réseau avec un timeout nul (traitement immédiat des paquets entrants), le second interroge `stdin` avec un court timeout permettant la saisie de commandes. Cette architecture permet au joueur de taper des commandes tout en recevant instantanément les coups de l'adversaire, simulant un parallélisme sans les risques inhérents à la programmation concurrente. La figure 7 en annexe illustre cette alternance temporelle.

3.4.3 Le patron Observateur pour le découplage

Interface de rappels. La communication entre le module réseau et les couches supérieures est régie par le patron *Observer*. À l'initialisation, le `NetworkManager` reçoit une structure `NetworkEvents` contenant neuf pointeurs de fonctions couvrant tous les événements significatifs : réception d'un coup, connexion ou déconnexion d'un joueur, réception d'une invitation, démarrage d'une partie, erreur protocolaire, expiration du temps de réflexion, et réception d'une liste de joueurs.

Inversion de contrôle. Lorsque le réseau reçoit un coup valide, il se contente d'invoquer le rappel `OnMoveReceived(move)` sans connaître l'identité de l'appelant. C'est la couche d'interface qui décide de la réaction appropriée : le CLI met à jour l'affichage textuel du plateau et réaffiche le prompt, tandis que la GUI redessine les cases concernées via les primitives GTK. Cette inversion de contrôle garantit que le module réseau reste totalement indépendant des interfaces

utilisateur : les fichiers d'implémentation CLI et GUI fournissent chacun leur propre jeu de rappels sans qu'une seule ligne du cœur réseau n'ait été modifiée.

3.4.4 Routage des paquets et arbitrage serveur

Architecture du routeur. Une fois un paquet désérialisé, il est acheminé vers un gestionnaire dédié via une structure de sélection indexée par le type de paquet. Cette conception respecte le principe ouvert/fermé : ajouter un nouveau type de message nécessite uniquement l'ajout d'un cas dans le gestionnaire, sans modifier la logique de routage existante.

```c
void handle_server_packet(NetworkManager *manager, ServerContext *ctx,
                          int client_index, Packet *pkt) {
  switch (pkt->type) {
    case C2S_JOIN:   process_join_packet(manager, ctx, client_index, pkt->payload); break;
    case C2S_INVITE: process_invite_packet(manager, ctx, client_index, pkt->payload); break;
    case C2S_MOVE:   process_move_packet(manager, ctx, client_index, pkt->payload); break;
    case C2S_HEARTBEAT: break;  // Mise a jour silencieuse du timestamp
    // ... autres cas
    default: printf("[SERVER] Unmanaged packet (Type: %d)\n", pkt->type);
  }
}
```

Listing 8 – Routage des paquets par type

Arbitrage anti-triche. Le serveur ne se contente pas de relayer aveuglément les coups entre joueurs. Avant tout transfert, il maintient une copie locale du plateau pour chaque partie active et valide la légalité de chaque coup reçu. Si un coup est déterminé illégal par le moteur de jeu, il est rejeté avec un message d'erreur explicite envoyé au client fautif, et l'adversaire n'est jamais notifié de cette tentative. Ce mécanisme garantit qu'un client malveillant ou bogué ne peut pas corrompre l'état de jeu de son adversaire ni obtenir un avantage indu.

```c
// Extrait de process_move_packet()
GameState *server_board = &ctx->active_games[game_idx];

if (is_move_legal(server_board, parsed_move)) {
  apply_move(server_board, parsed_move);  // MAJ plateau arbitre
  // Relais vers l'adversaire...
} else {
  printf("[ANTI-CHEAT] Client %d tried illegal move\n", client_index);
  send_error(manager, client_index, "Illegal move detected!");
}
```

Listing 9 – Validation anti-triche des coups

Routage par identifiant de partie. Chaque partie active possède un identifiant unique attribué séquentiellement par le serveur. Lorsqu'un client envoie un coup, le serveur consulte son champ `game_id` pour identifier la partie concernée, puis parcourt les clients connectés pour trouver celui partageant le même identifiant avec un statut « en jeu ». Cette recherche, bien que linéaire dans le pire cas ($\mathcal{O}(n)$ où n est le nombre de clients), reste négligeable en pratique car le nombre de clients actifs est borné.

3.4.5 Sérialisation et framing TCP

Format des paquets. Chaque paquet suit le format textuel `"TYPE|PAYLOAD\n"`. La sérialisation et désérialisation s'effectuent en temps linéaire par rapport à la taille du message :

```c
int SerializePacket(const Packet *pkt, char *buffer_out, size_t max_len) {
  return snprintf(buffer_out, max_len, "%d|%s\n", pkt->type, pkt->payload);
}
```

Listing 10 – Sérialisation des paquets

Extraction des messages. Le tampon TCP accumule les fragments jusqu'à réception d'un délimiteur `'\n'`. L'extraction utilise `memchr()` pour la recherche et `memmove()` pour la compaction :

```c
int tcp_buffer_extract_line(tcp_buffer_t *buf, char *out_msg, size_t max_len) {
  char *newline = memchr(buf->data, '\n', buf->length);
  if (newline == NULL) return EXTRACT_NO_MSG;

  size_t msg_len = newline - buf->data;
  memcpy(out_msg, buf->data, msg_len);
  out_msg[msg_len] = '\0';
```

```c

  size_t remaining = buf->length - msg_len - 1;
  memmove(buf->data, newline + 1, remaining);  // Compaction
  buf->length = remaining;
  return EXTRACT_SUCCESS;
}
```

Listing 11 – Extraction d'une ligne du tampon TCP

3.5 Le module d'interface graphique (GUI)

L'architecture de l'interface graphique a été pensée pour séparer la logique métier de la présentation visuelle, s'inspirant du patron Modèle-Vue-Contrôleur (MVC).

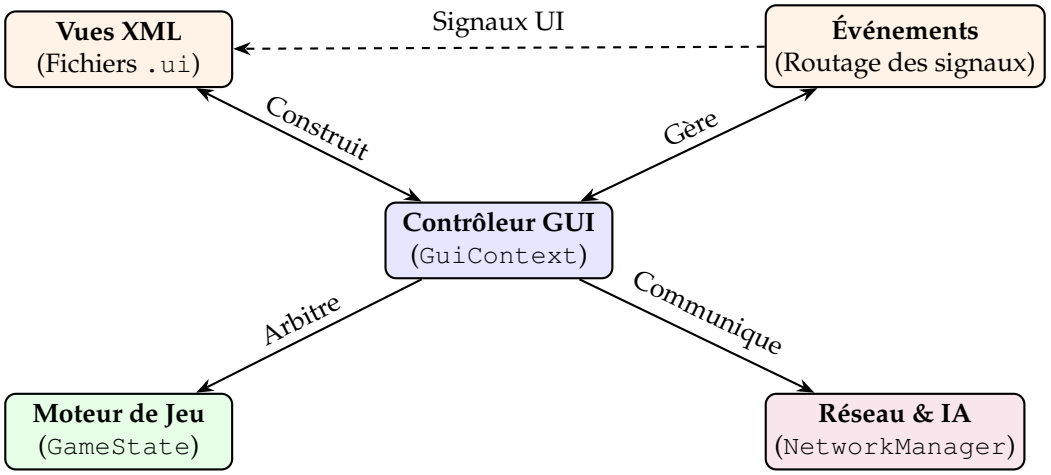


FIGURE 5 – Architecture du module graphique : le contrôleur orchestre les vues asynchrones, le réseau et le modèle de jeu.

3.5.1 Séparation vue/logique via XML

L'interface visuelle est définie dans des fichiers de description déclarative (XML). Cette séparation stricte entre définition de l'interface et logique procédurale présente deux avantages : la modification de la mise en page d'un écran est indépendante de la logique applicative, et l'initialisation du code se concentre purement sur la connexion des actions (*event binding*). Les ressources graphiques sont compilées dans un binaire embarqué au démarrage (*Resource Bundle*), rendant l'exécutable autonome.

3.5.2 Le contrôleur asynchrone et la Main Loop

Le point central de l'architecture GUI est son contrôleur. Le système de fenêtrage reposant sur une boucle d'événements bloquante (*Main Loop*), toute boucle d'attente classique (pour écouter le réseau ou calculer un coup d'IA) gèlerait l'interface. Le contrôleur résout ce problème en intégrant l'écoute réseau et l'IA sous forme de tâches asynchrones planifiées par le séquenceur de l'interface (*timeouts*). Réseau, IA et rendu graphique coopèrent ainsi de manière fluide sur le même fil d'exécution.

3.5.3 Navigation avec GtkStack

La structure de navigation repose sur un conteneur dynamique gérant un empilement de vues (menu principal, configuration, jeu, lobby réseau). Le couplage lâche, permis par le stockage de balises de destination directement dans les propriétés des boutons, permet de définir la navigation globale depuis les vues XML, sans multiplier les fonctions de routage spécifiques dans le contrôleur.

3.5.4 Le plateau de jeu et la validation retardée

Le déplacement d'une pièce suit un protocole strict en trois étapes : sélection de la pièce, déplacement temporaire en mémoire locale pour assurer le retour visuel, puis confirmation finale par le tir de la flèche. La méthode de mutation officielle de l'état (`apply_move`) n'est sollicitée qu'à l'issue de cette transaction. Cette approche transactionnelle garantit que le moteur de jeu central (et son historique d'annulation) n'enregistre jamais d'états partiels ou de coups incomplets.

3.6 Module CLI

Le fichier de commandes CLI constitue la base de l'interface en ligne de commande. Sa fonction principale reçoit le texte saisi par le joueur et utilise une série de comparaisons pour identifier l'action demandée. Des constantes symboliques remplacent systématiquement les longueurs de chaînes codées en dur, ce qui facilite la maintenance lors de modifications des noms de commandes.

Pour éviter les variables globales, l'ensemble du contexte est centralisé dans une structure `CliContext` transmise à chaque fonction. Cette structure regroupe le pointeur vers la partie en cours, les réglages, l'état de navigation (menu principal, partie réseau, statut absent), le gestionnaire de réseau, ainsi que des métadonnées de gestion du temps et de l'IA. Ce contexte permet par exemple de bloquer la commande de déplacement si le joueur se trouve en lobby réseau, ou d'empêcher le lancement d'un second serveur si un premier est déjà actif.

3.7 Architecture des contrôleurs

Les contrôleurs CLI et GUI partagent une responsabilité commune : orchestrer les interactions entre l'utilisateur, le moteur de jeu, l'IA et le réseau. Cependant, leurs modèles d'exécution diffèrent fondamentalement.

3.7.1 Le contrôleur CLI : boucle événementielle hybride

Le contrôleur CLI implémente une boucle principale non bloquante qui doit surveiller simultanément deux sources d'événements indépendantes : le réseau et le clavier. Plutôt que de créer deux threads, nous utilisons un double multiplexage avec `poll()` :

```c
void run_interactive_mode(game_config_t *config) {
  CliContext *ctx = calloc(1, sizeof(CliContext));
  NetworkManager_Init(&ctx->net_manager, create_cli_network_events(ctx));

  // Configuration de readline en mode callback
  rl_callback_handler_install("", linehandler);

  struct pollfd fds[1];
  fds[0].fd = STDIN_FILENO;
  fds[0].events = POLLIN;

  while (ctx->game_running) {
    // 1. Tick reseau (poll interne avec timeout=0)
    NetworkManager_Tick(ctx->net_manager);

    // 2. Verification fin de partie
    if (is_game_over(ctx->current_game, ctx->current_game->current_player)) {
      handle_game_over(ctx);
      continue;
    }

    // 3. Tour de l'IA si applicable
    if (is_it_ai_turn(ctx->config, ctx->current_game->current_player)) {
      execute_ai_turn(ctx);
    }

    // 4. Gestion du timeout reseau synchronise
    handle_network_timeout(ctx);

    // 5. Poll clavier avec timeout court (100ms)
    int poll_res = poll(fds, 1, POLL_STDIN_TIMEOUT_MS);
    if (poll_res > 0 && (fds[0].revents & POLLIN)) {
      rl_callback_read_char();
    }
  }

  cleanup_resources(ctx);
}
```

Listing 12 – Boucle principale du contrôleur CLI

Cette architecture permet au joueur de taper des commandes tout en recevant instantanément les coups de l'adversaire, simulant un parallélisme sans les risques de la programmation concurrente.

3.7.2 Dispatch des commandes CLI

Les commandes saisies sont routées via une fonction de dispatch centralisée. Le contrôleur vérifie d'abord le contexte actuel (menu principal, lobby réseau, partie en cours) avant d'exécuter l'action :

```c
void handle_command(const char *input, CliContext *ctx) {
  // Commandes toujours disponibles
  if (strncmp(input, "help", 4) == 0) {
    execute_help(ctx);
    return;
  }
  if (strncmp(input, "quit", 4) == 0) {
    execute_quit(ctx);
    return;
  }

  // Commandes dependantes du contexte
  if (ctx->in_main_menu) {
    // Seules new, load, server start, join sont autorisees
    if (strncmp(input, "new", 3) == 0) {
      execute_new_game(ctx);
    } else if (strncmp(input, "server start", 12) == 0) {
      execute_server_start(ctx, input + 12);
    }
    // ...
  } else if (ctx->in_network_match) {
    // En partie reseau : uniquement les coups et commandes de jeu
    if (is_move_notation(input)) {
      execute_network_move(ctx, input);
    }
  } else {
    // Partie locale : toutes les commandes de jeu
    handle_local_game_command(ctx, input);
  }
}
```

Listing 13 – Extrait du dispatch des commandes CLI

3.7.3 Le contrôleur GUI : intégration asynchrone avec GLib

Le contrôleur graphique s'intègre à la boucle d'événements GTK (*Main Loop*) via des callbacks et des timeouts planifiés. L'IA ne peut pas calculer de manière bloquante car cela gèlerait l'interface :

```c
void guiHandlePlayerMove(AppContext *ctx, Move move) {
  if (!is_move_legal(ctx->gameState, move)) {
    g_print("Error: Engine rejected the move.\n");
    return;
  }

  apply_move(ctx->gameState, move);
  clearHighlights(ctx);
  drawBoard(ctx);
  ctx->currentPhase = PHASE_MOVE_QUEEN;

  if (is_game_over(ctx->gameState, ctx->gameState->current_player)) {
    show_game_over_dialog(ctx);
    return;
  }

  // Planification asynchrone du tour IA
  if (is_it_ai_turn(ctx->gameConfig, ctx->gameState->current_player)) {
    // Delai de 50ms pour laisser le rendu se terminer
    g_timeout_add(50, guiAIPlayTurn, ctx);
  }
}

static gboolean guiAIPlayTurn(gpointer user_data) {
  AppContext *ctx = (AppContext *)user_data;

  AI_Interface *ai = get_current_ai(ctx);
  Move ai_move = ai->vtable->get_best_move(ai, *(ctx->gameState));

  guiHandlePlayerMove(ctx, ai_move);

  return G_SOURCE_REMOVE;  // Execution unique
}
```

Listing 14 – Intégration asynchrone de l'IA dans le contrôleur GUI

Le callback `guiAIPlayTurn` est exécuté par le séquenceur GLib après le délai spécifié, permettant au rendu graphique de se terminer avant le calcul de l'IA.

4 Explication approfondie

4.1 Fonctionnalité réseau : Système d'invitations et machine à états (F40)

4.1.1 Prérequis et problème

Le cahier des charges (F40) impose un système complet d'invitations permettant aux joueurs de se défier dans le lobby. Ce système doit gérer plusieurs cas complexes : un joueur peut être indisponible (`AWAY`), déjà en partie (`INGAME`), ou en attente de réponse (`WAITGAME`). Les invitations doivent expirer automatiquement après 5 minutes, et toute transition d'état invalide doit être rejetée avec un message explicite.

4.1.2 Solution implémentée : machine à états finis

Nous avons modélisé le cycle de vie d'un client par un automate fini à cinq états. Chaque transition est gardée par des vérifications explicites :

```
typedef enum {
  CLIENT_STATUS_CONNECTED,  // TCP etabli, attente JOIN
  CLIENT_STATUS_IDLE,       // Dans le lobby, disponible
  CLIENT_STATUS_AWAY,       // Indisponible temporairement
  CLIENT_STATUS_WAITGAME,   // Invitation en attente
  CLIENT_STATUS_INGAME      // Partie en cours
} ClientStatus;
```

Listing 15 – Énumération des statuts clients

Traitement d'une invitation. Lorsqu'un joueur envoie une invitation, le serveur vérifie l'éligibilité via des gardes d'état :

```
// Extrait de process_invite_packet()
if (ctx->clients[sender_index].status != CLIENT_STATUS_IDLE) {
  send_error(manager, sender_index, "You must be IDLE to invite.");
  return;
}
if (ctx->clients[target_id].status == CLIENT_STATUS_AWAY) {
  send_error(manager, sender_index, "Player is currently AWAY.");
  return;
}
// Transitions d'etat apres validation
ctx->clients[sender_index].status = CLIENT_STATUS_WAITGAME;
ctx->clients[target_id].pending_invite_from = sender_index;
```

Listing 16 – Gardes de transition pour les invitations

Expiration automatique. À chaque tick serveur, les invitations pendantes sont vérifiées. Une invitation non répondue après 300 secondes déclenche le retour au statut `IDLE` pour les deux joueurs avec notification explicite.

4.1.3 Garanties de cohérence

Cette implémentation garantit trois propriétés : **déterminisme** (chaque paquet produit exactement une transition ou un rejet), **isolation** (un joueur en partie ne peut recevoir d'invitations), et **récupération automatique** (les timeouts libèrent les ressources bloquées).

4.2 Fonctionnalité 4 : Approfondissement itératif avec contrainte de temps

4.2.1 Prérequis et problème

L'algorithme Minimax classique explore l'arbre de jeu jusqu'à une profondeur fixe d, puis retourne le meilleur coup trouvé. Cette approche présente un défaut majeur : si d est trop grand, le calcul peut dépasser le temps imparti, ne retournant aucun coup valide. Si d est trop petit, l'IA joue de manière myope. Le cahier des charges (F34) impose donc un mécanisme garantissant le retour du **meilleur coup trouvé à la profondeur maximale complétée** avant l'expiration du délai.

4.2.2 Solution implémentée : Iterative Deepening

L'approfondissement itératif consiste à explorer l'arbre en profondeur croissante ($d = 1, 2, 3, \ldots$) jusqu'à épuisement du temps. À chaque itération complète, le meilleur coup est mis à jour. Si le temps expire en milieu d'exploration, le coup de l'itération précédente est retourné.

Implémentation. La fonction `minimax_get_best_move` contient une boucle principale :

```
Move best_move_overall = available_moves[0];

for (int d = 1; d <= data->max_depth; d++) {
    if (get_time_ms() - start_time >= data->time_limit)
        break;

    // Tri des coups pour placer le meilleur de l'iteration
    // precedente en tete (ameliore le pruning)
    if (d > 1)
        sort_moves(count, best_move_overall, available_moves);

    Move move_at_depth = run_root_search(data, &root_state, d,
                                         start_time, available_moves, count);

    // Sauvegarde uniquement si l'iteration s'est terminee
    if (get_time_ms() - start_time < data->time_limit) {
        best_move_overall = move_at_depth;
    }
}

return best_move_overall;
```

Listing 17 – Boucle d'approfondissement itératif

Optimisation par réordonnancement des coups. Avant chaque nouvelle profondeur, la fonction `sort_moves` place le meilleur coup de l'itération précédente en tête du tableau. Cette heuristique exploite le fait que le coup optimal à profondeur $d-1$ est souvent proche de celui à profondeur d, ce qui améliore significativement l'efficacité de l'élagage alpha-beta en explorant d'abord les branches prometteuses.

Garanties temporelles. La vérification `get_time_ms() - start_time < data->time_limit` est effectuée :

— Avant chaque nouvelle itération de profondeur
— À l'intérieur de `alpha_beta_forward` avant chaque expansion de nœud
— Avant la sauvegarde du meilleur coup

Cette triple vérification garantit que l'algorithme ne dépasse jamais le temps alloué de plus de quelques millisecondes (le temps d'une seule expansion de nœud).

4.2.3 Intégration avec Forward Pruning (F33)

Lorsque le forward pruning est activé, chaque itération de profondeur bénéficie d'un faisceau de recherche adaptatif. La largeur du faisceau croît avec la profondeur restante :

```
int get_beam_width(int maximizing, int current_depth, int max_depth) {
    int base = maximizing ? NB_BEST_MOVES_PLAYER : NB_BEST_MOVES_OPPONENT;
    int depth_bonus = (max_depth - current_depth) * 3;
    return base + depth_bonus;
}
```

Ainsi, à profondeur 1, seuls 10 coups sont explorés (joueur maximisant), mais à profondeur 5 avec $d_{\max} = 5$, le faisceau s'élargit à $10 + (5-1) \times 3 = 22$ coups. Cette approche équilibre vitesse (petits faisceaux en profondeur) et précision (faisceaux larges près de la racine).

4.3 Fonctionnalité 3 : le système de glisser-déposer à deux phases dans la GUI

4.3.1 Contrainte : atomicité d'un tour en deux gestes

Un tour aux Amazones est indivisible : une reine se déplace *puis* une flèche est tirée en une seule action logique. Du point de vue de l'interface, deux gestes utilisateur distincts (un glisser-déposer et un clic) doivent être combinés en une seule transaction avant que la fonction officielle d'application du coup ne soit appelée. Le défi consiste à maintenir un état intermédiaire cohérent entre les deux gestes sans corrompre l'état officiel du moteur de jeu.

4.3.2 Solution : modification temporaire du plateau

L'approche retenue consiste à modifier temporairement le plateau en mémoire lors du dépôt de la reine, sans passer par la fonction d'application officielle. La pièce est déplacée visuellement dans la structure d'état, les deux extrémités du déplacement sont mémorisées dans le contexte applicatif, et la phase passe au tir de la flèche. En cas d'annulation par clic droit, la pièce est restaurée à sa position d'origine et la phase revient au déplacement, sans aucune trace dans le moteur de jeu. Lors de la confirmation du tir, la modification temporaire est d'abord annulée pour restaurer l'état d'avant le déplacement, puis la fonction `apply_move()` est appelée avec la structure `Move` complète. Cela garantit que le moteur (et sa pile d'annulation) ne voit jamais un état partiel.

4.3.3 Intégration de l'IA via la boucle d'événements GLib

Lorsqu'une partie opposant un humain à l'IA est lancée, l'IA ne peut pas calculer son coup de façon bloquante dans le thread principal de la vue (cela gèlerait l'interface). Le tour de l'IA est donc déclenché de façon asynchrone via un appel planifié avec un délai minimal de 500 millisecondes. Ce délai donne au rendu graphique le temps de terminer l'animation avant que l'IA ne joue, offrant un retour visuel perceptible à l'utilisateur.

5 Revue critique du projet

5.1 Ce qui a été accompli

5.1.1 Module IA

L'intégralité du cahier des charges a été respectée. Sur le plan des performances, notre MCTS, particulièrement avec l'accélération OpenMP, se révèle extrêmement robuste, offrant un excellent niveau de jeu. L'architecture a tenu la charge, permettant de faire s'affronter différentes heuristiques de manière fluide.

5.1.2 Module réseau

Le module réseau remplit la quasi-totalité de son cahier des charges (98 %) en offrant une architecture robuste, performante et sécurisée. Il exploite une approche bi-protocole alliant UDP pour la découverte automatique des serveurs et TCP pour la fiabilité des échanges de jeu. Le serveur, conçu de manière événementielle et mono-threadée autour de l'appel système `poll()`, est capable de gérer jusqu'à 64 clients et 32 parties simultanées. Ses performances s'appuient sur une empreinte mémoire strictement déterministe grâce à des pools d'objets pré-alloués et un accès aux données en temps constant. Le cycle de vie des joueurs est rigoureusement encadré par une machine à états à cinq phases, couplée à un système d'invitations complet et à un battement de cœur silencieux qui prévient les déconnexions abusives. De plus, le serveur s'impose comme un véritable arbitre : il maintient une copie des plateaux, valide systématiquement la légalité des coups pour bloquer toute tentative de triche, et gère l'attribution des victoires par forfait en cas d'abandon ou de coupure.

5.1.3 Module GUI

L'interface graphique couvre la totalité du cycle de jeu local : navigation entre les pages, configuration des paramètres (taille du plateau, modes IA, raccourcis clavier reconfigurables, langue), partie locale contre un autre joueur ou contre l'IA, avec annulation et rejeu de coups, indication visuelle des cases accessibles et dialogue de fin de partie. Le plateau interactif supporte aussi bien le glisser-déposer que le clic. La charte graphique est uniforme et entièrement paramétrable via la feuille de style CSS. Des fenêtres modales permettent d'accéder aux configurations et informations sans quitter la page de jeu.

5.2 Limitations et améliorations possibles

5.2.1 Module IA

Actuellement, le Minimax ne peut dépasser une profondeur de deux dans un temps raisonnable sans élagage anticipé. S'il atteint la profondeur cinq avec le pruning, cette approche montre ses limites face à l'heuristique QDKD : l'IA a tendance à écarter des coups de blocage critiques car ils ne semblent pas optimaux à court terme. Quelques comportements indésirables persistent sur des cas aux limites très spécifiques (contrainte de temps nulle, ou états terminaux imprévus).

Côté améliorations, des tables de transposition avec hachage Zobrist constitueraient la priorité absolue pour gagner en profondeur. Rendre le Minimax multi-threadé offrirait un gain immédiat. Côté MCTS, l'ajout du réseau de neurones est un premier pas, mais le modèle (régression linéaire simple) manque de complexité et de données d'entraînement. Une approche hybride combinant MCTS en début de partie et un solver de fin de partie permettrait de résoudre les petits plateaux de manière déterministe.

5.2.2 Module réseau

Le système présente certaines limites liées à ses partis pris architecturaux initiaux. Le modèle mono-threadé, bien qu'excellent pour les entrées/sorties, paralyserait l'intégralité du serveur s'il devait absorber des tâches bloquantes, comme le calcul centralisé d'une intelligence artificielle. Sur le plan des données, la persistance du tableau des scores via un simple fichier texte reste vulnérable aux corruptions en cas de crash, et l'absence de chiffrement expose le trafic en clair sur le réseau. De plus, l'expérience utilisateur souffre de l'absence d'un mécanisme de reconnexion : la moindre micro-coupure entraîne une exclusion définitive de la partie. Enfin, quelques finitions manquent à l'appel, notamment l'interrogation du statut du serveur à distance sur les clients.

Pour faire évoluer ce module vers un standard de production, on à identifiés plusieurs améliorations techniques majeures. L'intégration d'une base de données SQLite garantirait l'atomicité et la sécurité des sauvegardes de scores. Pour pallier les limites du mono-thread, l'introduction d'un pool de *worker threads* permettrait de déléguer les calculs

lourds tout en gardant la boucle principale réactive. La sécurité des échanges serait assurée par une encapsulation TLS (via OpenSSL), tandis qu'un système de jetons de session (UUID) offrirait aux joueurs une fenêtre de tolérance pour se reconnecter de manière transparente après une chute de réseau. Enfin, pour soutenir une montée en charge massive, le remplacement de `poll()` par `epoll()` couplé à une table de hachage pour le routage des coups permettrait de dépasser la limite actuelle des 64 clients tout en conservant des performances optimales.

5.2.3 Module GUI

La page multijoueur expose les boutons d'hébergement et de connexion à un serveur, mais ceux-ci ne sont pas encore connectés au module réseau dans la GUI : le mode multijoueur reste accessible uniquement via l'interface CLI. Les boutons de sauvegarde et chargement de parties sont présents dans l'interface mais leurs gestionnaires de signaux ne sont pas implémentés. La taille des cellules du plateau est fixée à 80×80 pixels : sur des plateaux de grande taille, le plateau déborderait de la fenêtre ; un calcul dynamique en fonction de la taille de la fenêtre constituerait une amélioration naturelle

Conclusion

La réalisation de ce projet en C nous a permis de mettre en pratique des concepts couvrant deux domaines complémentaires : la programmation système (gestion des sockets POSIX, multiplexage asynchrone, protocoles applicatifs) et la programmation d'interfaces graphiques (GTK4, boucle d'événements GLib, glisser-déposer, CSS). L'architecture en couches fortement découplées — moteur de jeu, réseau, CLI, GUI — s'est révélée être un choix pertinent : le même moteur de jeu et la même API réseau ont été exploités par les deux interfaces sans modification. L'utilisation du patron Observateur pour les événements réseau a notamment permis de connecter la couche graphique au module réseau sans introduire de dépendance circulaire, ce qui constitue l'un des succès architecturaux majeurs de ce projet.

Références

[1] W. Zamkauskas. El Juego de las Amazones. *El Acertijo*, vol. 4, 1992.

[2] The GNOME Project. *GTK 4 Reference Manual*. `https://docs.gtk.org/gtk4/`, 2021–2024. Consulté en 2024.

Table des matières

1 Introduction **1**
- 1.1 Historique . . . 1
- 1.2 Règles . . . 1
- 1.3 Complexité . . . 2

2 Algorithmes et structures de données **2**
- 2.1 Le cœur du jeu et l'Intelligence Artificielle . . . 2
  - 2.1.1 Représentation de l'état : les Bitboards . . . 2
  - 2.1.2 Analyse de Complexité des Opérations Bitboards . . . 3
  - 2.1.3 L'approche Minimax et ses heuristiques . . . 3
  - 2.1.4 Monte Carlo Tree Search (MCTS) et apprentissage automatique . . . 4
- 2.2 Réseau : structures de données et protocole . . . 4
  - 2.2.1 Le problème du *Framing* TCP . . . 5
  - 2.2.2 Protocole applicatif et sérialisation . . . 5
  - 2.2.3 Structures de contexte et gestion des connexions . . . 5
- 2.3 Contrôleurs : structures de données et contextes . . . 6
  - 2.3.1 Le contexte CLI `CliContext` . . . 6
  - 2.3.2 Le contexte GUI `AppContext` . . . 6
- 2.4 Moteur de jeu : structures de données . . . 7
  - 2.4.1 Modélisation matricielle de l'état du jeu . . . 7
  - 2.4.2 Architecture Bitboard . . . 7
- 2.5 Moteur de jeu : algorithmes . . . 7
  - 2.5.1 Algorithme de validation des trajectoires (Raycasting) . . . 7
  - 2.5.2 Piles d'historique et algorithme d'annulation (Undo/Redo) . . . 7
- 2.6 Interface graphique : structures de données . . . 7
  - 2.6.1 Le contexte applicatif `GuiContext` . . . 7
  - 2.6.2 L'automate des phases de jeu . . . 8
  - 2.6.3 Les raccourcis clavier `KeyBindings` . . . 8

3 Architecture du projet **8**
- 3.1 Découpage en modules . . . 8
  - 3.1.1 Diagramme de dépendances . . . 8
- 3.2 Architecture du module IA . . . 8
  - 3.2.1 Le patron Factory et la table virtuelle . . . 8
  - 3.2.2 Injection de dépendances par ConfigMap . . . 9
  - 3.2.3 Découplage évaluateurs et sélecteurs . . . 10
- 3.3 Le moteur de jeu . . . 10
- 3.4 Architecture du module réseau . . . 10
  - 3.4.1 La façade `NetworkManager` . . . 10
  - 3.4.2 Le multiplexage asynchrone avec `poll()` . . . 11
  - 3.4.3 Le patron Observateur pour le découplage . . . 11
  - 3.4.4 Routage des paquets et arbitrage serveur . . . 12
  - 3.4.5 Sérialisation et framing TCP . . . 12
- 3.5 Le module d'interface graphique (GUI) . . . 13
  - 3.5.1 Séparation vue/logique via XML . . . 13
  - 3.5.2 Le contrôleur asynchrone et la Main Loop . . . 13
  - 3.5.3 Navigation avec GtkStack . . . 13
  - 3.5.4 Le plateau de jeu et la validation retardée . . . 13
- 3.6 Module CLI . . . 14
- 3.7 Architecture des contrôleurs . . . 14
  - 3.7.1 Le contrôleur CLI : boucle événementielle hybride . . . 14
  - 3.7.2 Dispatch des commandes CLI . . . 14
  - 3.7.3 Le contrôleur GUI : intégration asynchrone avec GLib . . . 15

4 Explication approfondie **16**
4.1 Fonctionnalité réseau : Système d'invitations et machine à états (F40) . . . 16
4.1.1 Prérequis et problème . . . 16
4.1.2 Solution implémentée : machine à états finis . . . 16
4.1.3 Garanties de cohérence . . . 16
4.2 Fonctionnalité 4 : Approfondissement itératif avec contrainte de temps . . . 16
4.2.1 Prérequis et problème . . . 16
4.2.2 Solution implémentée : Iterative Deepening . . . 16
4.2.3 Intégration avec Forward Pruning (F33) . . . 17
4.3 Fonctionnalité 3 : le système de glisser-déposer à deux phases dans la GUI . . . 17
4.3.1 Contrainte : atomicité d'un tour en deux gestes . . . 17
4.3.2 Solution : modification temporaire du plateau . . . 17
4.3.3 Intégration de l'IA via la boucle d'événements GLib . . . 18

5 Revue critique du projet **18**
5.1 Ce qui a été accompli . . . 18
5.1.1 Module IA . . . 18
5.1.2 Module réseau . . . 18
5.1.3 Module GUI . . . 18
5.2 Limitations et améliorations possibles . . . 18
5.2.1 Module IA . . . 18
5.2.2 Module réseau . . . 18
5.2.3 Module GUI . . . 19

A Liste des besoins généraux **21**

B Liste des besoins gestion du jeu **22**

C Liste des besoins CLI **23**

D Liste des besoins format de sauvegarde **23**

E Liste des besoins structures de données **24**

F Liste des besoins GUI **24**

G Liste des besoins intelligence artificielle **25**

H Liste des besoins réseau **27**

I Synthèse globale du projet **30**

J Protocole réseau : types de paquets **31**

K Schéma réseaux complets **33**

A Liste des besoins généraux

Exigence CDC	Compl.	Implémentation
F1–F3. Lancement et configuration du programme		
F1. Lancement via exécutable `amazons [OPTIONS]`	**100%**	Exécutable `amazons` compilé via CMake ; options `-h`, `-V`, `-v`, `-d` implémentées dans `main.c`.
F2. Fichier de configuration `.amazonsrc` au format INI	**100%**	`config.c` lit/écrit le fichier INI ; sections `[defaults]` ; options CLI supplantent la config.
F3. Internationalisation (i18n) anglais/français	**100%**	`gettext` avec fichier `amazons.pot` ; détection via `LANG`/`LC_ALL` ; fallback anglais.

TABLE 2 – F1–F3 : Lancement et configuration — **100%**

Exigence CDC	Compl.	Implémentation
F4–F6. Options de lancement		
F4. Interface CLI par défaut, chargement fichier en argument	**100%**	`cli_controller.c` lance le shell interactif; `load_game()` restaure depuis fichier argument.
F5. Mode blitz avec `-b/-blitz` et `-t TIME/-time TIME`	**100%**	Variables `white_time_left`, `black_time_left`; `update_blitz_timer()` appelé à chaque coup ; défaut 30 min.
F6. Mode contest avec `-c/-contest`	**100%**	Lecture position depuis fichier ; réponse coup sur stdout ; mode non-interactif.

TABLE 3 – F4–F6 : Options de lancement — **100%**

Exigence CDC	Compl.	Implémentation
F7–F9. Modes de jeu		
F7. Interface graphique avec `-g/-gui`	**100%**	`gui_application.c` avec GTK4; mêmes fonctionnalités que CLI via fenêtre graphique.
F8. Mode IA avec `-a [COLOR]/-ai [COLOR]`	**100%**	`ai_white_cfg` et `ai_black_cfg` dans `game_config_t`; couleurs W/B/A supportées.
F9. Taille plateau avec `-n SIZE/-size SIZE` (4–11)	**100%**	`board_size` configurable; validation `MIN_BOARD_SIZE` à `MAX_BOARD_SIZE`; défaut 10.

TABLE 4 – F7–F9 : Modes de jeu — **100%**

B Liste des besoins gestion du jeu

Exigence CDC	Compl.	Implémentation
F10–F14. Gestion des règles et de la partie		
F10. Gestion des règles : validation coups, détection fin de partie	**100%**	`is_move_legal()` valide via bitboards; `is_game_over()` détecte blocage; `get_winner()` détermine vainqueur.
F11. Gestion des erreurs utilisateur avec messages explicites	**100%**	Coups illégaux rejetés par le moteur ; contrôleur affiche message d'erreur localisé.
F12. Undo/redo avec historique des coups	**100%**	Piles LIFO `undo_stack/redo_stack`; `undo_move()` et `redo_move()` avec inversion atomique ; annulation jusqu'au joueur humain précédent.
F13. Conseils de coup via commande `hint`	**100%**	`sync_from_logic()` expose l'état au module IA ; meilleur coup affiché en notation algébrique.
F14. Gestion du blitz : affichage temps, pause, arrêt partie	**75%**	Affichage temps restant et arrêt automatique implémentés ; **mise en pause non implémentée**.

TABLE 5 – F10–F14 : Gestion des règles — **95%**

C Liste des besoins CLI

Exigence CDC	Compl.	Implémentation
F15–F17. Shell interactif		
F15. Shell avec prompt, commandes `new`, `help`, `quit`, `load`, `save`, `pause`, `hint`, `undo`, `redo`, `show`, `set`	**100%**	`cli_commands.c` implémente toutes les commandes ; dispatch via table de fonctions ; messages d'erreur explicites.
F16. Proposer sauvegarde avant `quit` si non sauvegardé	**100%**	Dialogue `Save the game before quitting? [y/N]` ; redemande en cas d'erreur d'enregistrement.
F17. Édition ligne de commande (insertion, suppression, curseur)	**100%**	GNU Readline intégré via `input.c` ; navigation curseur avec flèches.

TABLE 6 – F15–F17 : Shell interactif — **100%**

Exigence CDC	Compl.	Implémentation
F18–F20. Historique et notation		
F18. Historique commandes avec flèches et `<Ctrl>+<R>`	**100%**	GNU Readline gère l'historique ; navigation haut/-bas ; recherche incrémentale.
F19. Complétion commandes avec `<Tab>`	**100%**	`rl_completion_entry_function` configuré ; double-tab affiche alternatives.
F20. Notation algébrique `e2-e4/e6` (colonnes a–k, lignes 1–11)	**100%**	Parsing dans `cli_commands.c` ; validation coordonnées ; message si coup illégal.

TABLE 7 – F18–F20 : Historique et notation — **100%**

D Liste des besoins format de sauvegarde

Exigence CDC	Compl.	Implémentation
F21–F25. Format de sauvegarde		
F21. Sauvegarde/restauration au format ASCII avec gestion erreurs	**100%**	`save_game()` et `load_game()` dans `save.c` ; parseur robuste avec messages d'erreur.
F22. Commentaires ligne (`#`) et bloc (`{}`)	**100%**	`clean_line()` filtre commentaires ; `in_block_comment` gère les blocs multi-lignes.
F23. Section `[settings]` avec paramètres clé=valeur	**100%**	`write_settings()` et `parse_settings_line()` ; debug, timeout, ai-mode, etc.
F24. Section `[game]` avec plateau ASCII (Q/q/X/_) et joueur courant (W/B)	**100%**	`write_game_state()` et `parse_board_line()` ; format conforme à la spec.
F25. Section `[history]` avec coups en notation algébrique	**100%**	`write_history()` et `parse_history_line()` ; format `e2-e4/e6`.

TABLE 8 – F21–F25 : Format de sauvegarde — **100%**

E Liste des besoins structures de données

Exigence CDC	Compl.	Implémentation
F26–F27. Module bitboard		
F26. Module bitboard indépendant pour représenter l'état du plateau	**100%**	Structure `Bitboard` 128 bits (`low`/`high`); `BitboardState` avec reines et flèches; module isolé dans `bitboard.c`/`bitboard.h`.
F27. Algorithmes bitboard : validation coups, calcul coups possibles, application coup, scores, fin de partie	**100%**	Kogge-Stone pour trajectoires; `expand_queen_slide()`/`expand_king_step()`; masques pré-calculés `MASK_NOT_A`/`MASK_NOT_H`; `bitboard_state_is_game_over()`.

TABLE 9 – F26–F27 : Module bitboard — **100%**

F Liste des besoins GUI

Exigence CDC	Compl.	Implémentation
F28–F30. Interface graphique		
F28. Menus File (New, Load, Save, Config, Info, Quit) et Game (Undo, Redo, Pause, Hint)	**90%**	Menu principal avec 5 boutons; pages via `GtkStack`; **Load Game non connecté en GUI**; popups Config et Info fonctionnels.
F29. Raccourcis clavier configurables (10 actions)	**100%**	`GtkShortcutController` avec actions `Ctrl+N/L/S/,/I/Q/U/R/P/H`; personnalisation via boutons toggle; sauvegarde dans config.
F30. Glisser-déposer pour jouer à la souris	**100%**	`GtkDragSource` + `GtkDropTarget` sur chaque cellule; `onDrag()`/`onDrop()` gèrent le déplacement; tir flèche par clic.

TABLE 10 – F28–F30 : Interface graphique — **97%**

Fonctionnalité GUI	Compl.	Implémentation
Fonctionnalités GUI complémentaires		
Plateau interactif dans `GtkGrid`	**100%**	`setupBoard()` alloue $n \times n$ cellules; `drawBoard()` met à jour le rendu.
Mise en évidence des cases accessibles	**100%**	Calcul via `bitboard_get_reachable()`; classe CSS `highlight-valid`.
Annulation déplacement par clic droit	**100%**	`GtkGestureClick` secondaire; `onRightClick()` restaure l'état.
Intégration IA asynchrone	**100%**	Tour IA planifié sans bloquer GTK; `guiHandlePlayerMove()` enchaîne.
Undo/redo double coup (joueur + IA)	**100%**	`undoButtonClick()` appelle `undo_move()` deux fois; uniquement en mode IA.
Charte graphique violet/or	**100%**	`style.css` chargé via `GtkCssProvider`.
Compatibilité GTK4 $\geq$ 4.10	**100%**	Blocs `#if GTK_CHECK_VERSION` pour `GtkAlertDialog`/`GtkDropDown`.
Libération ressources à la fermeture	**100%**	`freeAppContext()` libère plateau et appelle `game_cleanup()`.

TABLE 11 – Fonctionnalités GUI complémentaires — **100%**

G Liste des besoins intelligence artificielle

Exigence CDC	Compl.	Implémentation
F31–F33. Algorithme Minimax		
F31. Temps de réflexion borné avec `-ai-time TIME`; retour meilleur coup trouvé (défaut 5s)	**100%**	`time_limit` vérifié via `get_time_ms()` dans `alpha_beta_conditions()` et `run_root_search()`; `DEFAULT_TIME_LIMIT = 5000`.
F32. Minimax avec $\alpha\beta$-pruning via `-ai-mode minimax`	**100%**	Coupure `beta <= alpha` dans `alpha_beta_forward()`; `update_best_move()` met à jour bornes.
F33. Au moins 3 fonctions d'évaluation via `-ai-minimax-scoring`	**100%**	`eval_me()` (mobilité), `eval_rth()` (territoire relatif BFS), `eval_qdkd()` (distance reine/roi combinée); sélection via `get_evaluator()`.

TABLE 12 – F31–F33 : Algorithme Minimax — **100%**

Exigence CDC	Compl.	Implémentation
F34–F35. Minimax avancé		
F34. Approfondissement itératif via `-ai-mode iterative`	**100%**	Boucle `for (d = 1; d <= max_depth; d++)` avec conservation meilleur coup; tri via `sort_moves()` pour améliorer pruning.
F35. Profondeur configurable via `-ai-minimax-depth DEPTH`	**100%**	Paramètre `max_depth` initialisé via `ConfigMap`; `DEFAULT_MAX_DEPTH = 3`; profondeur auto si non spécifiée.

TABLE 13 – F34–F35 : Minimax avancé — **100%**

Exigence CDC	Compl.	Implémentation
F36–F37. MCTS et Machine Learning		
F36. MCTS avec UCT via `-ai-mode mcts`	**100%**	`select_node()` avec interface `ISelector`; `expand()`, `rollout()`, `backpropagate()`; UCT avec $C = \sqrt{2}$; support multi-threading OpenMP.
F37. Fonction de sélection ML via `-ai-mcts-selection ML`	**25%**	Architecture `ISelector` extensible; intégration TensorFlow C API prévue; **modèle non entraîné**; fonctionnalité expérimentale.

TABLE 14 – F36–F37 : MCTS et Machine Learning — **63%**

Fonctionnalité bonus	Compl.	Implémentation
Améliorations IA au-delà du cahier des charges		
Forward pruning pour réduction espace de recherche	**100%**	`get_beam_width()` calcule largeur faisceau selon profondeur; `get_top_moves()` filtre coups prometteurs.
Parallélisation MCTS multi-threads	**100%**	Arbres indépendants par thread via `#pragma omp parallel`; fusion résultats via `merge_and_get_best()`.
Rollout heuristique avec évaluation anticipée	**100%**	`rollout()` coupe à profondeur 5; `get_mcts_win_probability()` estime probabilité victoire.
Factory pattern pour instanciation IA	**100%**	`ai_factory.c` crée Minimax ou MCTS selon config; extensible pour nouveaux algorithmes.

TABLE 15 – Fonctionnalités bonus IA

Exigence du cahier des charges	Complétion	Commentaire
F31–F33 : Algorithme Minimax de base	**100%**	3 évaluateurs implémentés
F34–F35 : Minimax avancé	**100%**	Iterative deepening fonctionnel
F36–F37 : MCTS et Machine Learning	**63%**	ML expérimental non entraîné
Total module IA	**90%**	—

TABLE 16 – Synthèse de la couverture des exigences IA

H Liste des besoins réseau

Exigence CDC (F38)	Compl.	Implémentation
F38. Découverte de serveurs et connexion client-serveur basique		
`server start [PORT]` : Démarrage serveur TCP sur port configurable (12345 par défaut), diffusion UDP de présence	**100%**	`NetworkManager_ServerStart();` option `-s [PORT];` broadcast UDP toutes les 10 s sur port 12346.
`server list` : Liste des serveurs via UDP broadcast, expiration après 30 s sans signal	**100%**	`ClientDiscoveryContext` écoute le port UDP; `NetworkManager_GetServerList()` filtre les entrées expirées après 30 s.
`join [IP[:PORT]]` : Connexion au serveur avec gestion d'erreur explicite	**100%**	`NetworkManager_ClientJoin()` + `NetworkManager_SendJoin();` messages d'erreur si connexion échoue.
`server stop` : Arrêt du serveur avec notification des clients connectés	**100%**	`NetworkManager_ShutdownServer()` envoie `S2C_ERROR` à tous les clients avant nettoyage.
`quit` : Déconnexion propre du client, retour en mode local	**100%**	`execute_quit()` appelle `NetworkManager_Cleanup();` retour au menu principal.
`ping` : Test de connexion avec affichage du temps de réponse	**100%**	Horodatage en ms dans le payload de `C2S_PING`; RTT calculé à réception de `S2C_PONG`.
Timeout de déconnexion (1 minute) pour gérer les déconnexions inopinées	**100%**	Boucle de vérification dans `handle_server_tick();` kick après 60 s d'inactivité. Heartbeat silencieux toutes les 30 s pour les clients légitimes inactifs.
Messages ASCII terminés par `'\n'`	**100%**	Format `"TYPE|PAYLOAD\n";` `SerializePacket()` / `DeserializePacket()`.

TABLE 17 – F38 : Découverte et connexion basique — **100%**

Exigence CDC (F39)	Compl.	Implémentation
F39. Serveur multi-clients et gestion de parties		
Gestion multi-clients simultanés (thread par client ou asynchrone)	**100%**	Modèle événementiel mono-threadé avec `poll()`; jusqu'à 64 clients simultanés sans threads.
Protection des données partagées (mutex/verrous)	**100%**	Architecture mono-threadée élimine les accès concurrents; aucun mutex nécessaire par conception.
`server status` : Port d'écoute, nombre de clients, parties en cours	**75%**	`NetworkManager_GetServerStatus()` fonctionne en mode serveur local; non exposé via paquet réseau pour un client distant.
`players` : Liste des joueurs (id, nom, statut)	**100%**	`NetworkManager_GetServerPlayerList()`; paquets `C2S_LIST` / `S2C_LIST`.
`scoreboard` : Tableau des scores persistant (victoires, défaites, parties)	**100%**	Fichier `scoreboard.txt` géré par `network_stats.c`; paquets `C2S_SCOREBOARD` / `S2C_SCOREBOARD`.
Routage des coups vers la bonne partie	**100%**	`process_move_packet()` identifie l'adversaire via `game_id` partagé et route le coup.
Validation des coups côté serveur (anti-triche)	**100%**	`is_move_legal()` appelé sur copie locale du plateau avant tout relais; coup illégal rejeté avec `S2C_ERROR`.
Option `-s [PORT]` / `-server [PORT]`	**100%**	Traitée dans `main`; lance directement en mode serveur.
Option `-d` / `-daemon` : Mode headless	**100%**	Appel à `daemon(0,0)` sous POSIX; serveur sans interface.

TABLE 18 – F39 : Serveur multi-clients — **97%**

Exigence CDC (F40)	Compl.	Implémentation
F40. Système d'invitations et gestion avancée des joueurs		
`players [PLAYER_ID]` : Informations détaillées d'un joueur spécifique	**100%**	Branche `target_id != -1` dans `NetworkManager_GetServerPlayerList()`.
Statuts de joueur : `idle`, `away`, `waitgame`, `ingame`	**100%**	Énumération `ClientStatus` à 5 états ; transitions protégées par garde-fous.
`new PLAYER_ID` : Envoi d'invitation avec notification	**100%**	`process_invite_packet()` vérifie disponibilité ; `S2C_INVITE` envoyé à la cible.
`accept` : Acceptation avec démarrage immédiat	**100%**	`process_accept_packet()` crée le jeu et envoie `S2C_GAME_START` aux deux joueurs.
`decline` : Refus avec notification de l'inviteur	**100%**	`process_decline_packet()` libère les deux joueurs ; `S2C_DECLINE` envoyé.
`cancel` : Annulation d'invitation en attente	**100%**	Paquet `C2S_CANCEL` ; le serveur notifie la cible via `S2C_CANCEL` et libère l'inviteur.
`away` / `back` : Changement de disponibilité	**100%**	Paquets `C2S_AWAY` / `C2S_BACK` ; transitions `IDLE` ↔ `AWAY`.
Timeout d'invitation (5 minutes) avec expiration automatique	**100%**	Boucle de vérification dans `handle_server_tick()` ; expiration après 300 s avec notification des deux joueurs.
Empêcher actions invalides (inviter joueur en partie, accepter sans invitation, etc.)	**100%**	Vérifications systématiques : `pending_invite_from == -1`, statut cible ≠ `INGAME`/`AWAY`, etc. ; `S2C_ERROR` explicite.
Validation unicité du pseudo à la connexion	**100%**	`process_join_packet()` vérifie l'unicité avant d'accepter ; `S2C_REJECT` si doublon.

TABLE 19 – F40 : Système d'invitations — **100%**

Fonctionnalité bonus	Compl.	Implémentation
Améliorations réseau au-delà du cahier des charges		
Heartbeat silencieux pour maintenir les connexions inactives légitimes	**100%**	Paquet `C2S_HEARTBEAT` émis toutes les 30 s ; permet aux joueurs `AWAY` de rester connectés.
Gestion du timeout de tour (60 s) avec notification réseau	**100%**	Timer dans la boucle CLI ; paquet `PKT_TIMEOUT` routé par le serveur à l'adversaire.
Victoire par forfait lors de déconnexion adverse	**100%**	`disconnect_server_client()` met à jour les statistiques et envoie `S2C_OPPONENT_LEFT`.
Empreinte mémoire déterministe et bornée	**100%**	Slots de taille fixe ; aucune allocation dynamique en cours de fonctionnement.

TABLE 20 – Fonctionnalités bonus réseau

Exigence du cahier des charges	Complétion	Commentaire
F38 : Découverte et connexion basique	**100%**	Toutes les commandes implémentées
F39 : Serveur multi-clients et gestion de parties	**97%**	`server status` limité au mode local
F40 : Système d'invitations et gestion avancée	**100%**	Machine à états complète
Total module réseau	**99%**	—

TABLE 21 – Synthèse de la couverture des exigences réseau

I Synthèse globale du projet

Module	Complétion	Points d'attention
F1–F9 : Généralités et options	**100%**	Toutes options implémentées
F10–F14 : Gestion du jeu	**95%**	Pause blitz à implémenter
F15–F20 : CLI	**100%**	Shell complet avec Readline
F21–F25 : Format de sauvegarde	**100%**	Parseur robuste
F26–F27 : Structures de données	**100%**	Bitboards optimisés
F28–F30 : GUI	**97%**	Load Game GUI à connecter
F31–F37 : Intelligence artificielle	**90%**	ML expérimental non entraîné
F38–F40 : Réseau	**99%**	`server status` limité
Total projet	**97%**	—

TABLE 22 – Synthèse globale de la couverture des exigences F1–F40

J Protocole réseau : types de paquets

Type	Direction	Description
Authentification et session		
C2S_JOIN	C → S	Demande de connexion avec pseudo
S2C_WELCOME	S → C	Confirmation d'acceptation du pseudo
S2C_REJECT	S → C	Rejet du pseudo (déjà utilisé)
C2S_HEARTBEAT	C → S	Maintien de connexion silencieux
C2S_PING	C → S	Test de latence (horodatage inclus)
S2C_PONG	S → C	Réponse au ping
Gestion du lobby		
C2S_LIST	C → S	Demande de liste des joueurs
S2C_LIST	S → C	Liste des joueurs connectés
C2S_AWAY	C → S	Passage en statut indisponible
C2S_BACK	C → S	Retour en statut disponible
C2S_SCOREBOARD	C → S	Demande du tableau des scores
S2C_SCOREBOARD	S → C	Ligne du tableau des scores
Système d'invitations		
C2S_INVITE	C → S	Invitation avec taille de plateau
S2C_INVITE	S → C	Notification d'invitation reçue
C2S_ACCEPT	C → S	Acceptation d'invitation
C2S_DECLINE	C → S	Refus d'invitation
S2C_DECLINE	S → C	Notification de refus
C2S_CANCEL	C → S	Annulation d'invitation émise
S2C_CANCEL	S → C	Notification d'annulation
Déroulement de la partie		
S2C_GAME_START	S → C	Démarrage avec rôle assigné
C2S_MOVE	C → S	Envoi d'un coup joué
S2C_MOVE	S → C	Relais du coup adverse
PKT_TIMEOUT	Bidir.	Signalement d'expiration de tour
C2S_GAME_OVER	C → S	Déclaration de fin de partie
S2C_OPPONENT_LEFT	S → C	Déconnexion de l'adversaire
S2C_ERROR	S → C	Message d'erreur générique

TABLE 23 – Liste complète des 27 types de paquets du protocole. C = Client, S = Serveur, Bidir. = Bidirectionnel.

K Schéma réseaux complets

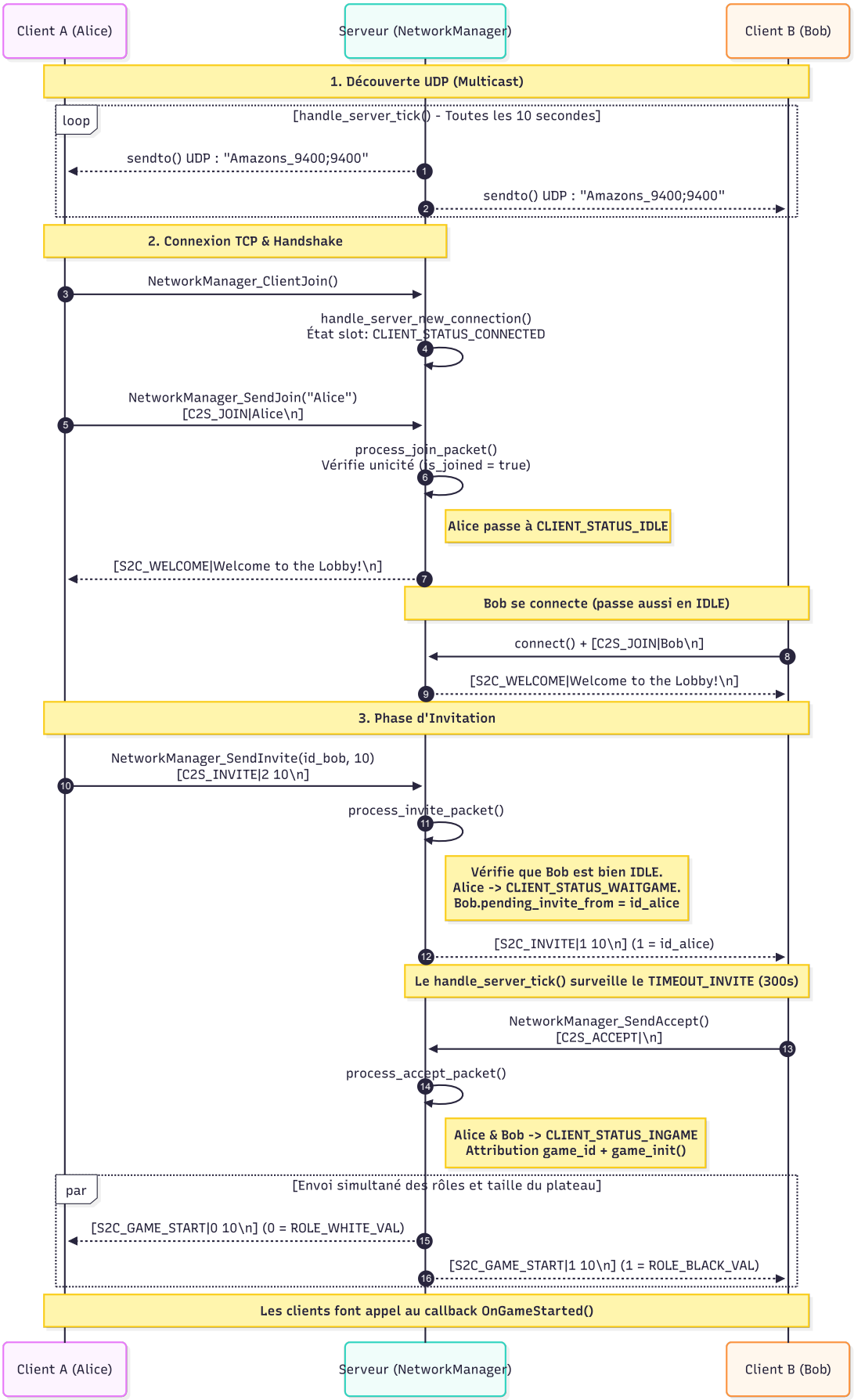


FIGURE 6 – Protocole d'établissement d'une partie : découverte UDP, authentification, invitation et démarrage synchronisé.

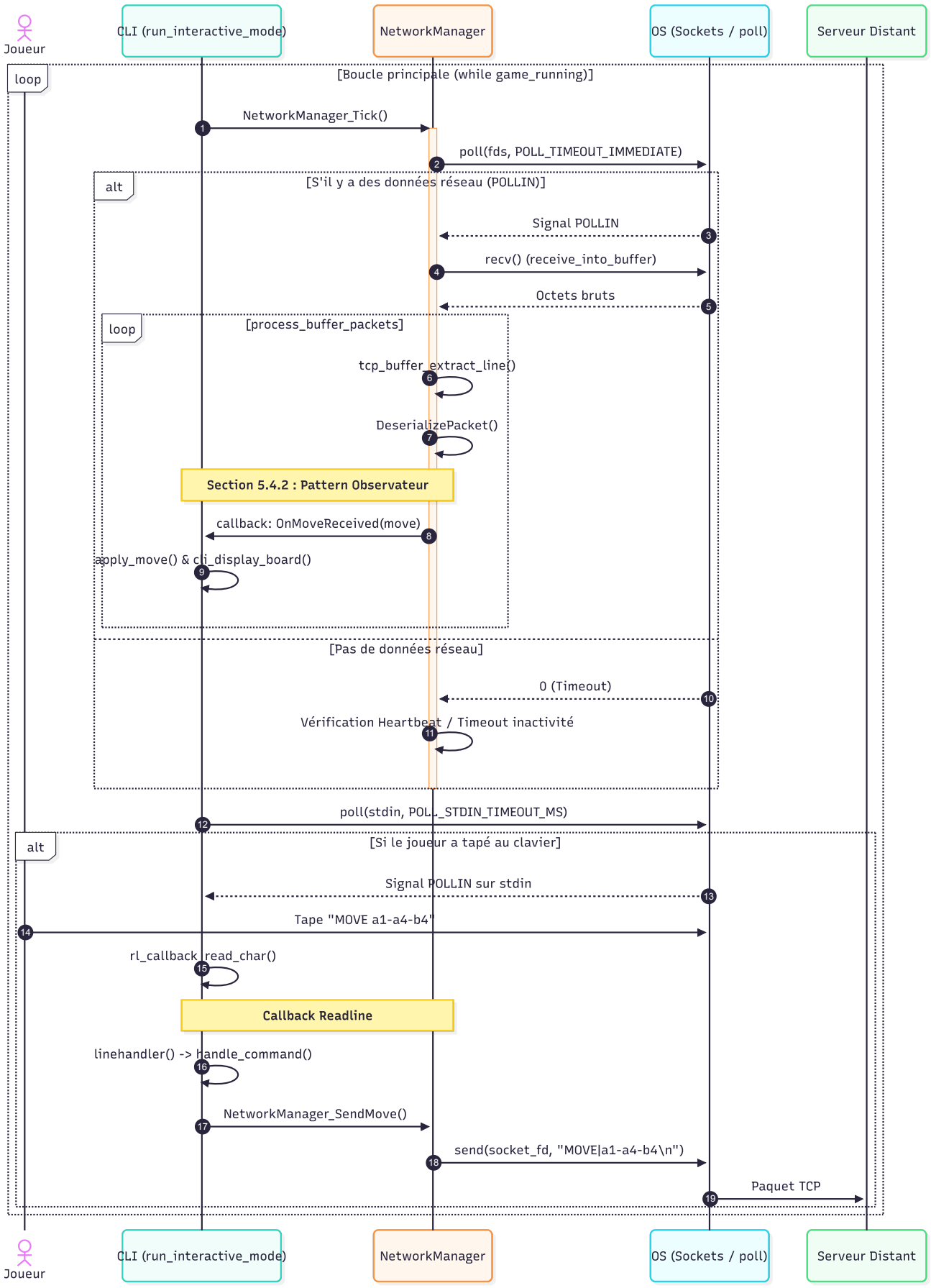


FIGURE 7 – Boucle principale asynchrone : alternance entre écoute réseau et lecture clavier via deux appels `poll()` non bloquants.